

0. Template

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
#define pb push_back
#define sz(a) ((int)a.size())
#define all(a) a.begin(), a.end()
#define fi first
#define se second
#define lb lower_bound
#define ub upper_bound
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pii;
// #include<ext/pb_ds/assoc_container.hpp>
// using namespace __gnu_pbds;
// using indexed_set = tree<int, null_type, less<int>, rb_tree_tag,
// tree_order_statistics_node_update>;

void solver(){

}

int main(){
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    // freopen("name.in", "r", stdin);
    // freopen("name.out", "w", stdout);
    int t = 1;
    // cin>>t;
    while(t--){
        solver();
    }
}
```

```
    return 0;
}
```

1. Math

1.1 Inverso Modular

```
ll mod_inverse(ll a, ll mod) {
    ll i = mod, v = 0, d = 1;
    while(a > 0) {
        ll t = i/a, x = a;
        a = i%x;
        i = x;
        x = d;
        d = v - t*x;
        v = x;
    }
    v %= mod;
    if(v < 0)
        v += mod;
    return v; // (a * v) % mod = 1
}

int main() {
    ll a, n;
    cin >> a >> n;
    cout << mod_inverse(a, n) << endl;
    return 0;
}
```

1.2 Exponenciación Binaria

```

ll myPow(ll b, ll e){
    ll result = 1;
    while(e > 0){
        if(e % 2 == 1)
            result = ((b % mod) * (result % mod)) % mod;
        b = ((b % mod) * (b % mod)) % mod;
        e /= 2;
    }
    return result;
}

```

1.3 Criba de Erastotenes

const int limite = 10001000; // Hasta este numero hay un poco mas de 6e5 primos

```

vector<bool> esPrimo(limite + 1, 1);
vector<int> primos;

```

```

void sieve(){
    esPrimo[0] = esPrimo[1] = 0;
    for(ll p = 2; p * p <= limite; p++){
        if(esPrimo[p]){
            for(ll i = p * p; i <= limite; i += p) esPrimo[i] = 0;
        }
    }

    for(int i = 2; i < limite; i++)
        if(esPrimo[i]) primos.pb(i);
}

```

1.4 GCD y LCM

```

ll gcd(ll a, ll b){

```

```

    ll temp;
    while(b != 0){
        temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

```

```

ll lcm(ll a, ll b){
    return (a / gcd(a, b)) * b;
}

```

1.5 Rectangular Partial Sums

```

const int maxn = 1001;
int a[maxn][maxn];
ll s[maxn][maxn];

```

```

void solver(){
    int n, m; cin >> n >> m;

    // La formula para calcular cada termino es:
    //  $S[i][j] = A[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]$ 
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> a[i][j];
            s[i][j] = (ll)a[i][j] + s[i-1][j] + s[i][j-1] - s[i-1][j-1];
        }
    }
}

```

// Para consultar cada rectangulo:
 // (y1, x1) es la esquina superior izquierda

```

// (y2, x2) es la esquina inferior derecha
// rectangulo = S[y2][x2] - S[y1-1][x2] - S[y2][x1-1] + S[y1-1][x1-1]
-1]
int q; cin>>q;
while(q--){
    int x1, y1, x2, y2; cin>>x1>>y1>>x2>>y2;
    cout<<s[y2][x2] - s[y1-1][x2] - s[y2][x1-1] + s[y1-1][x1-1]<<endl;
}
}

```

1.6 Random

```

//Esta función genera numeros aleatorios hasta de 64 bits
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());

```

1.7 Coeficientes Binomiales

```

// Complejidad  $O(n^2)$ 
//  $\text{binom}[i][j] = (i!) / (j! * (i - j)!)$ 

```

```

const int N = 501;
const ll mod = 1e9 + 7;
ll binom[N][N];

void calcBinom(){
    for(int i = 0; i < N; i++) binom[i][0] = binom[i][i] = 1;
    for(int i = 1; i < N; i++){
        for(int j = 1; j < i; j++){
            binom[i][j] = (binom[i-1][j-1] + binom[i-1][j]) % mod;
        }
    }
}

```

1.8 Multiplicación de Matrices

```

typedef vector<vector<long long>> matrix;

matrix multMatrix (const matrix &A, const matrix &B) {
    int len = A.size();
    matrix result(len, vector<long long>(len, 0));
    for (int i = 0; i < len; i++) {
        for (int j = 0; j < len; j++) {
            for (int k = 0; k < len; k++)
                result[i][j] = (result[i][j] + A[i][k] * B[k][j]) % mod;
        }
    }
    return result;
}

```

1.9 Potencia de Matrices

```

matrix potMatrix (matrix A, long long expo) {
    int len = A.size();
    matrix result(len, vector<long long>(len, 0));
    for (int i = 0; i < len; i++)
        result[i][i] = 1;
    while (expo > 0) {
        if (expo % 2 == 1)
            result = multMatrix(result, A);
        A = multMatrix(A, A);
        expo /= 2;
    }
    return result;
}

```

1.10 Números Prácticos

```

set<long long> getDivisors(long long n) {
    set<long long> divisors;
    for (long long i = 1; i * i <= n; ++i)
        if (n % i == 0)
            divisors.insert({i, n / i});
    return divisors;
}

```

```

bool isPractical (long long n) {
    set<long long> divisors = getDivisors(n);
    long long sum = 0;
    for (long long divisor : divisors) {
        if (divisor > sum + 1)
            return false;
        sum += divisor;
    }
    return true;
}

```

```

int main() {
    long long m;
    int t; cin >> t;
    while (t--) {
        cin >> m;
        cout << (isPractical(m) ? "Yes" : "No") << endl;
    }
}

```

/* Cabe resaltar que todos los multiples de 6 son número prácticos, además de que el único impar que es número práctico es el 1 */

1.11 Teorema Chino del Residuo

```
vector<ll> a, b;
```

```

ll mod_inverse(ll a, ll n) {
    ll i = n, v = 0, d = 1;
    while(a > 0) {
        ll t = i/a, x = a;
        a = i%x;
        i = x;
        x = d;
        d = v - t*x;
        v = x;
    }
    v %= n;
    if(v < 0)
        v += n;
    return v;
}

```

```

ll chinese_reminder(int len, ll B) {
    ll x = 0;
    //vector<ll> m(len), c(len);
    for(int i = 0; i < len; i++) {
        ll m = B/b[i];
        x += (a[i] * m * mod_inverse(m, b[i])) % B;
        //En caso de ser necesarios los vectores m[i] y c[i]
        // m[i] = B/b[i];
        // c[i] = m[i] * mod_inverse(m[i], b[i]);
        // x += (a[i] * c[i]) % B;
    }
}

```

```
return x%B;
```

```
}
```

```
int main () {
```

```
    ios_base::sync_with_stdio(0);cin.tie(NULL);
```

```
    int len; cin >> len;
```

```
    ll s1, s2;
```

```
    ll B = 1, x;
```

```
    for(int i = 0; i < len; i++) {
```

```
        cin >> s1 >> s2;
```

```
        a.push_back(s1); b.push_back(s2);
```

```
        B *= s2;
```

```
    }
```

```
    x = chinese_reminder(len, B);
```

```
    cout << x << endl;
```

```
    return 0;
```

```
}
```

2. Grafos

2.1 DSU

2.1.1 Union by Rank

```
const int maxn = 200005;
int components;
vector<int> leader(maxn);
vector<int> sets[maxn];

void initDSU(int n){
    components = n;
    for(int i = 1; i <= n; i++){
        leader[i] = i;
        sets[i].push_back(i);
    }
}

void join(int u, int v){
    int leaderU = leader[u], leaderV = leader[v];
    if(leaderU != leaderV){
        if(sets[leaderV].size() > sets[leaderU].size())
            swap(leaderU, leaderV);

        for(int i = 0; i < sets[leaderV].size(); i++){
            int v = sets[leaderV][i];
            leader[v] = leaderU;
            sets[leaderU].push_back(v);
        }
        sets[leaderV].clear();
        components--;
    }
}
```

2.1.2 Compresion de Caminos

```
const int maxn = 200005;
int components;
vector<int> graph[maxn];
vector<int> leader(maxn), sizen(maxn);

// Usando compresion de caminos y union by rank la
// complejidad se ve reducida a la inversa
// de Ackermann (casi tiempo constante)
// Si utilizamos compresion de caminos sin union by rank, la
// complejidad promedio es  $O(\lg n)$ 
// Lo he testeado para bosques sin union by rank y no hay
// problemas con el tiempo, no estoy
// seguro para cualquier tipo de grafo

void initDSU(int n){
    components = n;
    for(int i = 1; i <= n; i++){
        leader[i] = i;
        sizen[i] = 1;
    }
}

int find(int u){
    if(leader[u] != u)
        leader[u] = find(leader[u]);
    return leader[u];
}

void join(int u, int v){
    int leaderU = find(u), leaderV = find(v);
    if(leaderU != leaderV){
        if(sizen[leaderV] > sizen[leaderU])
```

```

        swap(leaderU, leaderV);
        leader[leaderV] = leaderU;
        sizen[leaderU] += sizen[leaderV];
        components--;
    }
}

```

```

void updateAllLeaders(int n){
    for(int i = 1; i <= n; i++){
        find(i);
    }
}

```

2.1.3 DSU Rollback

// Esta implementación usando Union-Find es ligeramente más rápida

```

const int maxn = 200005;
int components;
vector<int> leader(maxn), sizen(maxn);

```

```

// - op = 0 para decir que llegamos al anterior persist
// - u es el vertice cuyo lider cambiamos
// - len es el tamaño del vertice hacia el cual enviamos el
    componente
// que lideraba u

```

// - from es el vertice que ahora va a liderar a u

```

struct change{
    bool op;
    int u, len, from;
};

```

```

void initDSU(int n){
    components = n;
}

```

```

for(int i = 1; i <= n; i++){
    leader[i] = i;
    sizen[i] = 1;
}
}

```

```

int find(int u){
    if(leader[u] == u) return u;
    return find(leader[u]); // Notar que no se está usando
    compresion de caminos
}

```

```

void join(int u, int v, stack<change>& st){
    u = find(u); v = find(v);
    if(u != v){
        if(sizen[v] > sizen[u])
            swap(u, v);
        leader[v] = u;
        st.push({1, v, sizen[u], u});
        sizen[u] += sizen[v];
        components--;
    }
}

```

```

void rollback(stack<change>& st){
    while(!st.empty()){
        bool op = st.top().op;
        int u = st.top().u;
        int len = st.top().len, from = st.top().from;
        st.pop();
    }
}

```

```

    if(!op) break; // Si !op quiere decir que llegamos al anterior
    persist

```

```

    sizen[from] = len; // Recuperar la talla que tenia from antes
    del join que estamos eliminando
    leader[u] = u; // Ahora u vuelve a ser su propio lider, como
    antes del join
    components++; // Siempre que deshago una union quiere
    decir que dividi un componente
}
}

```

```

void persist(stack<change>& st){
    st.push({0, 0, 0, 0});
}

```

```

void solver(){
    int n, m; cin>>n>>m;
    initDSU(n);
    stack<change> st;
    while(m--){
        string op; cin>>op;
        if(op == "persist") persist(st);
        else if(op == "rollback"){
            rollback(st);
            cout<<components<<endl;
        }
        else{
            int u, v; cin>>u>>v;
            join(u, v, st);
            cout<<components<<endl;
        }
    }
}

```

```

    }
}

```

2.1.4 Dynamic Conectivity

```

const int maxn = 300005;
int components;
vector<int> leader(maxn), sizen(maxn);
vector<int> ans;

```

```

struct change{
    bool op;
    int u, len, from;
}; stack<change> st;

```

```

struct query{
    bool op;
    int t, l, r, u, v;
};

```

```

void initDSU(int n){
    components = n;
    for(int i = 1; i <= n; i++){
        leader[i] = i;
        sizen[i] = 1;
    }
}

```

```

int find(int u){
    if(leader[u] == u) return u;
    return find(leader[u]);
}

```



```

void join(int u, int v){
    u = find(u); v = find(v);
    if(u != v){
        if(sizen[v] > sizen[u])
            swap(u, v);
        leader[v] = u;
        st.push({1, v, sizen[u], u});
        sizen[u] += sizen[v];
        components--;
    }
}

```

```

void rollback(){
    while(!st.empty()){
        bool op = st.top().op;
        int u = st.top().u;
        int len = st.top().len, from = st.top().from;
        st.pop();

        if(!op) break;

        sizen[from] = len;
        leader[u] = u;
        components++;
    }
}

```

```

void persist(){
    st.push({0, 0, 0, 0});
}

```

```

// Para un problema en el cual ademas de las operaciones join y
// get
// nos piden remove para eliminar una arista, podemos
// aprovechar que
// tenemos todas las queries en adelante y resolverlo offline.
Cada
// arista va a tener un rango [l, r), que representa el tiempo desde
// se agrega la arista hasta que se elimina (en caso de que alguna
// arista no sea eliminada en ningun momento, asignaremos su
// valor r
// como m + 1 en 1-index o m en 0-index). Y cada operacion del
// tipo
// get va a tener un tiempo t. Para resolver el problema vamos a
// dividir
// cada segmento en mitades hasta que lleguemos a un
// segmento de tamaño
// 1, en ese caso vamos a revisar cuales queries get coinciden
// con el
// valor del rango y las vamos a responder. Cada que ingresamos
// a un
// segmento guardamos el DSU como estaba antes de hacer
// cualquier
// modificacion y cuando terminamos de analizar ese segmento
// devolvemos
// el DSU a su estado anterior
void work(int l, int r, queue<query>& q1){
    if(l == r - 1){
        persist();

        queue<query> q2;
        while(!q1.empty()){
            query q = q1.front(); q1.pop();

```

```

        if(!q.op && q.t == l) q2.push(q);
        if(q.op && q.l <= l && q.r >= r) join(q.u, q.v);
    }
    while(!q2.empty()){
        query q = q2.front(); q2.pop();
        ans[q.t] = components;
    }
    rollback();
    return;
}

persist();
queue<query> q2;
while(!q1.empty()){
    query q = q1.front(); q1.pop();
    if(q.op && q.l <= l && q.r >= r) join(q.u, q.v);
    // Si [l, r] pertenecen al rango de la arista, hacemos join
    else if((!q.op && q.t >= l && q.t < r) ||
        (q.op && ((q.l >= l && q.l < r) || (q.r - 1 >= l && q.r <= r))))
        q2.push(q);
    // Si la operacion es de arista y su rango se intersecta con [l,
    r), o si la
    // operacion es de tipo get y t pertenece a [l, r) agregamos esa
    querie a q2
    // para analizarlas en los segmentos que van a surgir al dividir
    [l, r)
}

int m = (l + r) >> 1;
queue<query> q3 = q2;
work(l, m, q2);
work(m, r, q3);

```

```

    rollback();
}

void solver(){
    int n, m; cin>>n>>m; if(!m) return;
    initDSU(n);
    map<pii, int> T;
    vector<query> queries;
    for(int i = 0; i < m; i++){
        char op; cin>>op;
        if(op == '?') queries.pb({0, i, 0, 0, 0, 0});
        else{
            int u, v; cin>>u>>v; if(u > v) swap(u, v);
            if(op == '+'){
                T[{u, v}] = queries.sz;
                queries.pb({1, 0, i, -1, u, v});
            }
            else queries[T[{u, v}]].r = i;
        }
    }

    queue<query> q1;
    for(int i = 0; i < queries.sz; i++){
        if(queries[i].op && queries[i].r == -1) queries[i].r = m;
        q1.push(queries[i]);
    }

    ans.resize(m);
    work(0, m, q1);
    for(int i = 0; i < m; i++){
        if(!ans[i]) continue;
        cout<<ans[i]<<endl;
    }
}

```

```

    }
}

```

2.1.5 Bipartition Test

```

const int maxn = 200005;
int components;
vector<int> leader(maxn), sizen(maxn), len(maxn);
// En el array len almacenamos la paridad de la distancia de
// un vertice v a su lider

// Este algortimo sirve para ir agregando aristas a un grafo
// de modo que siga siendo bipartito, hasta que llegue un punto
// en el que sea imposible que el grafo siga siendo bipartito
// Source: https://codeforces.com/edu/course/2/lesson/7/2

// Implementar DSU modificado para llenar el vector "len"
void initDSU(int n){
    for(int i = 1; i <= n; i++){
        leader[i] = i;
        sizen[i] = 1;
        len[i] = 0;
    }
}

pii find(int u){
    if(leader[u] == u) return {u, 0};

    pii val = find(leader[u]);
    leader[u] = val.first;
    len[u] = (len[u] + val.second) % 2;
    return {leader[u], len[u]};
// Retorna el lider de u y la paridad de la distancia al lider

```

```

}

```

```

void join(int u, int v){
    pii valU = find(u), valV = find(v);
    if(valU.first != valV.first){
        if(sizen[valU.first] > sizen[valV.first])
            swap(valU, valV);
        leader[valU.first] = valV.first;
        len[valU.first] = (1 + valU.second + valV.second) % 2;
        // La distancia entre el lider de u y el lider de v es igual
        // a la distancia entre u y su lider, mas la distancia entre
        // v y su lider, y una arista entre u y v
        sizen[valV.first] += sizen[valU.first];
    }
}

void updateAllLeaders(int n){
    for(int i = 1; i <= n; i++)
        find(i);
}

void solver(){
    int n, m; cin >> n >> m;
    initDSU(n);
    for(int i = 0; i < m; i++){
        int u, v; cin >> u >> v;
        join(u, v);
    }
}

```

2.2 MST

2.2.1 Kruskal

```
// Algoritmo de Kruskal para hallar el Minimal Spanning Tree
// Complejidad  $O(m * \lg(m))$ 
// En grafos densos  $m \approx n^2$ 
ll kruskal(vector<edge>& edges, int n){
    ll total = 0;
    sort(edges.begin(), edges.end());
    initDSU(n);

    for(int i = 0; i < edges.sz; i++){
        int u = edges[i].u, v = edges[i].v; ll w = edges[i].w;
        if(find(u) != find(v)){
            total += w;
            join(u, v);
        }
    }

    return total; // Retornar el peso total del MST
}
```

2.2.2 Prim

```
// Comparador para ordenar la cola de prioridad por el peso
menor
struct comp{
    bool operator() (pil a, pil b){
        return a.second > b.second; // Para Maximal Spanning Tree
        cambiar '>' por '<'
    }
};
```

// Algoritmo de Prim para hallar el Minimal Spanning Tree de un grafo no dirigido y conexo

// Complejidad $O(m * \lg(n))$

```
void prim(int source, int n, vector<ll>& d, vector<int>& parent){
    vector<bool> inQueue(n+1, 1);
    for(int i = 1; i <= n; i++){
        d[i] = inf;
        parent[i] = -1;
    }
    d[source] = 0;
    priority_queue<pil, vector<pil>, comp> pq;
    pq.push({source, 0});
    while(!pq.empty()){
        int u = pq.top().first;
        pq.pop();
        if(!inQueue[u]) continue;
        inQueue[u] = 0;
        for(pil edge : graph[u]){
            int v = edge.fi; ll w = edge.se;
            if(inQueue[v] && d[v] > w){
                d[v] = w;
                parent[v] = u;
                pq.push({v, d[v]});
            }
        }
    }
}
```

2.3 Dijkstra

```
const int maxn = 1e5 + 1;
const ll inf = 1e15;
vector<vector<pil>> graph(maxn);
```

```

struct comp{
    bool operator() (pil a, pil b){
        return a.second > b.second;
    }
};

// Complejidad O(m*log(n))
void dijkstra(int source, int n, vector<ll>& d, vector<int>& parent){
    for(int i = 1; i <= n; i++){
        d[i] = inf;
        parent[i] = -1;
    }
    d[source] = 0;
    priority_queue<pil, vector<pil>, comp> pq;
    pq.push({source, 0});
    while(!pq.empty()){
        int u = pq.top().first; ll w1 = pq.top().second;
        pq.pop();
        if(d[u] < w1) continue;
        for(pil edge : graph[u]){
            int v = edge.first; ll w2 = edge.second;
            if(d[v] > w1 + w2){
                d[v] = w1 + w2;
                parent[v] = u;
                pq.push({v, d[v]});
            }
        }
    }
}

```

2.4 Floyd-Warshall

```

const int maxn = 501, inf = 1e9;
int d[maxn][maxn]; // d[i][j] = Longitud de la ruta mas corta desde
i hasta j

```

```

// Complejidad O(n³)
// Sirve tanto para grafos dirigidos como no dirigidos
// Esta implementación es 1-index
void floyd(int n){
    for(int k = 1; k <= n; k++){
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                // Este if solo es necesario en caso de hayan pesos
                negativos
                if (d[i][k] < inf && d[k][j] < inf)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
}

```

// Algunas aplicaciones:

// - Safest Path:

// Maximizar la probabilidad de sobrevivir por algún camino

// Los valores son porcentajes

// $d[i][j] = \max(d[i][j], d[i][k] * d[k][j]);$

2.5 SPFA

```

const int maxn = 1e5 + 1;
const ll inf = 1e15;
vector<vector<pil>> graph(maxn);

```

```
// Complejidad O(n * m)
/*
    Sirve cuando tenemos pesos negativos y detecta si hay ciclos
    negativos.
    Si bien su complejidad en el peor de los casos es la misma que
    en Bellman-Ford,
    suele ser mejor para un grafo promedio.
*/
```

```
// Implementacion normal:
bool spfa(int s, int n, vector<ll>& d){
    for(int i = 1; i <= n; i++) d[i] = inf;
    d[s] = 0;
    vector<int> count(n+1);
    vector<bool> in(n+1);
    in[s] = 1;
    queue<int> q;
    q.push(s);

    while(!q.empty()){
        int u = q.front(); q.pop();
        in[u] = 0; // Marca que el vertice ya no esta en la cola

        for(pil edge : graph[u]){
            int v = edge.fi; ll w = edge.se;

            if(d[v] > d[u] + w){
                d[v] = d[u] + w;
                if(!in[v]){ // Antes de agregar un vertice a la cola, verifica
que no se encuentre ya en ella
                    in[v] = 1; // Marca que el vertice ahora esta en la cola
                    q.push(v);
```

```
                count[v]++;
                if(count[v] > n)
                    return 0; // Si el nodo ha sido metido en la cola más
de n veces, hay un ciclo negativo
            }
        }
    }

    return 1; // No hay ciclos negativos
}
```

2.6 BFS 0-1

```
const int maxn = 2e5 + 1, inf = 1e9;
vector<vector<int>> graph(maxn);

void BFS(int s, int n){
    deque<pii> q; q.push_front({1, 0});
    vector<int> d(n+1, inf); d[s] = 0;

    while(!q.empty()){
        int u = q.front().fi, dis = q.front().se; q.pop_front();
        if(dis > d[u]) continue;

        for(int v : graph[u]){
            int add = 0;
            // if(cond) add++;
            if(d[v] > d[u] + add){
                d[v] = d[u] + add;
                if(!add) q.push_front({v, d[v]});
                else q.push_back({v, d[v]});
            }
        }
    }
}
```

```

    }
  }
}

```

2.7 Finding Bridges

```

const int maxn = 2e5 + 5;
vector<vector<int>> graph(maxn);
vector<bool> visited(maxn);
vector<int> dp(maxn), sons[maxn], father(maxn);
set<pii> used;

```

```

void DFS(int u, int prev){
    visited[u] = 1;
    for(int v : graph[u]){
        if(v == prev) continue;
        if(visited[v]){
            if(used.find({v, u}) != used.end()) continue;
            dp[u]++;
            dp[v]--;
            used.insert({u, v});
            continue;
        }
        sons[u].pb(v);
        used.insert({u, v});
        father[v] = u;
        DFS(v, u);
    }
}

```

```

int calc(int u){
    for(int v : sons[u]) dp[u] += calc(v);
    return dp[u];
}

```

```

}

```

```

void solver(){
    int n, m; cin>>n>>m;
    for(int i = 0; i < m; i++){
        int u, v; cin>>u>>v;
        graph[u].pb(v);
        graph[v].pb(u);
    }
}

```

```

// 1 es la raiz en este caso
// Tener en cuenta que si el grafo no es conexo y se quieren
hallar puentes
// en algun componente, tocaria lanzar un DFS para cada
componente y en la
// iteracion final saltarse todos los vertices desde los cuales se
lanzo algun
// DFS
DFS(1, -1); // Crear DFS tree
calc(1); // Calcular el dp

for(int i = 2; i <= n; i++){
    // Si dp[i] == 0, la arista que conecta i y su padre son un
puente
    if(!dp[i]) cout<<"Bridge: "<<father[i]<<" - "<<i<<endl;
}
}

```

2.8 Floyd

```

const int maxn = 2e5 + 5;
vector<int> adj(maxn);

```

```

int floyd(int x){
    int a = adj[x];
    int b = adj[adj[x]];

    // At this point, the pointer a has walked k steps and the pointer
    b has walked
    // 2k steps, so the length of the cycle divides k
    while(a != b){
        a = adj[a];
        b = adj[adj[b]];
    }

    // the first node that belongs to the cycle can be found by
    // moving the pointer a to node x and advancing the pointers
    // step by step until they meet again
    a = x;
    while(a != b){
        a = adj[a];
        b = adj[b];
    }
    int firstCycleNode = a;

    // After this, the length of the cycle can be calculated as
    follows:
    b = adj[b];
    int length = 1;
    while(a != b){
        b = adj[b];
        length++;
    }

    return length;
}

```

```

}

```

2.9 LCA

```

const int maxn = 2e5 + 1;
const int LOG = 19; //El logaritmo base 2 del numero maximo de
nodos
vector<vector<int>> graph(maxn), up(maxn, vector<int>(LOG)); //
up[a][i] guarda el ancestro que está a 2^i distancia de a
vector<int> depth(maxn, -1);

```

/* Este algoritmo sirve para calcular el minimo ancestro comun (Lowest Common Ancestor) entre dos nodos especificos.

Cada query tiene una complejidad de $O(\lg(n))$, siendo n el número de nodos del grafo, gracias al Binary Lifting.

El algoritmo toma un nodo en especifico como raíz y calcula la profundidad de cada nodo según el número minimo de nodos que debe atravesar para llegar al nodo raíz, esto se calcula mediante un BFS. */

// Esta implementación sirve para grafos no dirigidos y dirigidos

```

void BFS(int s) {
    depth[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, s});
    while(!pq.empty()) {
        int v = pq.top().se;
        pq.pop();
        for(int u : graph[v]) {
            if(depth[u] == -1) {

```



```

    depth[u] = depth[v] + 1;
    up[u][0] = v;

    for(int j = 1; j < LOG; j++)
        up[u][j] = up[up[u][j-1]][j-1];
    pq.push({depth[u], u});
}
}
}

int LCA (int a, int b) {
    if(depth[a] < depth[b])
        swap(a, b);
    int k = depth[a] - depth[b];
    // Este ciclo pone a y b en el mismo nivel
    for(int j = LOG - 1; j >= 0; j--) {
        if(k & (1 << j))
            a = up[a][j];
    }
    if(a == b)
        return a;
    for(int j = LOG - 1; j >= 0; j--) {
        if(up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}

```

2.10 Minimum Vertex Cover

```

const int maxn = 2e5;

vector<vector<int>>> tree(maxn);
int dp[maxn][2];

// Un Vertex Cover es un subconjunto de vértices  $C \subseteq V$  tal
// que cada arista del grafo tiene al menos un extremo en C.
// Un minimum vertex cover nos devuelve el tamaño mínimo de C
// para que se cumpla esa condición

// Complejidad:
// Para un grafo cualquiera este es un problema NP - Completo,
// sin embargo, para un árbol o un bipartito, la complejidad
// es  $O(n)$ 
void DFS(int n, int parent){
    // 1 quiere decir que el vértice está en el cover, 0 que no está
    dp[n][0] = 0; dp[n][1] = 1;
    for(int i : tree[n]){
        if(i == parent) continue;
        DFS(i, n); // Calculamos la respuesta para cada subárbol de n
    }

    // Si n está en el cover, sus hijos pueden estar o no,
    // mientras que si n no está en el cover, todos sus
    // hijos tienen que estar
    for(int i : tree[n]){
        if(i == parent) continue;
        dp[n][0] += dp[i][1];
        dp[n][1] += min(dp[i][0], dp[i][1]);
    }
}

```

2.11 Tarjan Strongly Connected Components

/* Complexity: $O(E + V)$

Tarjan's algorithm for finding strongly connected components.

*d[i] = Discovery time of node i. (Initialize to -1)

*low[i] = Lowest discovery time reachable from node i. (Doesn't need to be initialized)

*scc[i] = Strongly connected component of node i. (Doesn't need to be initialized) (0-index)

*s = Stack used by the algorithm (Initialize to an empty stack)

*stacked[i] = True if i was pushed into s. (Initialize to false)

*ticks = Clock used for discovery times (Initialize to 0)

*current_scc = ID of the current_scc being discovered (Initialize to 0)

*/

```
const int maxn = 1e5 + 1;
```

```
vector<int> graph[maxn];
```

```
vector<int> d(maxn, -1), low(maxn), scc(maxn), reps(maxn, 0);
```

```
vector<bool> stacked(maxn);
```

```
stack<int> s;
```

```
int ticks = 0, current_scc = 0;
```

```
void tarjan(int u){
```

```
    d[u] = low[u] = ticks++;
```

```
    s.push(u);
```

```
    stacked[u] = true;
```

```
    const vector<int> &out = graph[u];
```

```
    for (int k=0, m=out.size(); k<m; ++k){
```

```
        const int &v = out[k];
        if (d[v] == -1){
            tarjan(v);
            low[u] = min(low[u], low[v]);
        }
        else if (stacked[v])
            low[u] = min(low[u], low[v]);
    }
    if (d[u] == low[u]){
        int v;
        do{
            v = s.top();
            s.pop();
            stacked[v] = false;
            scc[v] = current_scc;
        }while (u != v);
        current_scc++;
    }
}
```

```
void solver(){
```

```
    int n, m; cin >> n >> m;
```

```
    for (int i = 1; i <= m; i++){
```

```
        int u, v; cin >> u >> v;
```

```
        graph[u].pb(v);
```

```
    }
```

```
    // Hallar los SCC
```

```
    for (int i = 1; i <= n; i++){
```

```
        if (d[i] == -1)
```

```
            tarjan(i);
```

```
    }
```

```

    cout<<current_scc<<endl;
}

```

2.12 TopoSort con BFS

```

const int maxn = 2e5 + 1;
vector<vector<int>> graph(maxn);
vector<int> inDegree(maxn);

```

// Sirve para hallar un "Topological Sort" en un DAG (Directed Acyclic Graph)
 // Un "Topological Sort", es una forma de ordenar los nodos dado que para cada

// arista dirigida de 'u' a 'v', 'u' aparece antes que 'v' en el orden
 // Esta implementacion se base en un BFS, por ende la complejidad es $O(n + m)$

```

void topoSort(int n, vector<int>& orden){
    queue<int> q;
    for(int i = 1; i <= n; i++){
        if(!inDegree[i]){
            // Si el nodo no tiene ningun inDegree, quiere decir que
            // puede ir al inicio del topoSort
            q.push(i);
            orden.pb(i);
        }
    }

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : graph[u]){
            inDegree[v]--; // Quitamos un inDegree a cada vertice que
            // puede ser visitado desde otro ya procesado

```

```

        if(!inDegree[v]){
            // Si su inDegree ahora es 0, quiere decir que ya fueron
            // procesados todos los nodos que lo
            // anteceden, por ende ahora puede ser agregado al
            // topoSort, y lo metemos a la cola
            q.push(v);
            orden.pb(v);
        }
    }
}

```

```

void solver(){
    int n, m; cin>>n>>m;
    for(int i = 0; i < m; i++){
        int u, v; cin>>u>>v;
        graph[u].pb(v);
        inDegree[v]++;
    }

    vector<int> ans;
    topoSort(n, ans);
    if(ans.sz < n) cout<<"IMPOSSIBLE"<<endl;
    // Si el topoSort no contiene todos los nodos, quiere decir que
    // no hay ninguna forma
    // valida de ordenarlos, es decir, el grafo dado no es un DAG
    // (Contiene algun ciclo)
    else{
        for(int i = 0; i < ans.sz; i++){
            if(i) cout<<' ';
            cout<<ans[i];
        }
    }
}

```

```

        cout<<endl;
    }
}

```

2.13 Finf Faces on Planar Graph

// Estructura Point con campos "x" y "y" y operaciones entre puntos

```

struct Point {
    ll x, y;

    Point(ll x_, ll y_): x(x_), y(y_) {}

    bool operator < (const Point& p) const {
        return tie(x, y) < tie(p.x, p.y);
    }

    Point operator - (const Point& p) const {
        return Point(x - p.x, y - p.y);
    }

    ll cross (const Point & p) const {
        return x * p.y - y * p.x;
    }

    ll cross (const Point & p, const Point & q) const {
        return (p - *this).cross(q - *this);
    }

    ll half () const {
        return ll(y < 0 || (y == 0 && x < 0));
    }
};

```

```

//Hallar las caras de un grafo plano
vector<vector<ll>> find_faces(vector<Point> vertices,
vector<vector<ll>> adj) {
    ll n = vertices.size();
    vector<vector<char>> used(n);
    for (ll i = 0; i < n; i++) {
        used[i].resize(adj[i].size());
        used[i].assign(adj[i].size(), 0);
        auto compare = [&](ll l, ll r) {
            Point pl = vertices[l] - vertices[i];
            Point pr = vertices[r] - vertices[i];
            if (pl.half() != pr.half())
                return pl.half() < pr.half();
            return pl.cross(pr) > 0;
        };
        sort(adj[i].begin(), adj[i].end(), compare);
    }
    vector<vector<ll>> faces;
    for (ll i = 0; i < n; i++) {
        for (ll edge_id = 0; edge_id < adj[i].size(); edge_id++) {
            if (used[i][edge_id]) {
                continue;
            }
            vector<ll> face;
            ll v = i;
            ll e = edge_id;
            while (!used[v][e]) {
                used[v][e] = true;
                face.push_back(v);
                ll u = adj[v][e];

```

```

    ll e1 = lower_bound(adj[u].begin(), adj[u].end(), v, [&](ll l, ll
r) {
    Point pl = vertices[l] - vertices[u];
    Point pr = vertices[r] - vertices[u];
    if (pl.half() != pr.half())
        return pl.half() < pr.half();
    return pl.cross(pr) > 0;
}) - adj[u].begin() + 1;
    if (e1 == adj[u].size()) {
        e1 = 0;
    }
    v = u;
    e = e1;
}
reverse(face.begin(), face.end());
ll sign = 0;
for (ll j = 0; j < face.size(); j++) {
    ll j1 = (j + 1) % face.size();
    ll j2 = (j + 2) % face.size();
    ll val = vertices[face[j]].cross(vertices[face[j1]],
vertices[face[j2]]);
    if (val > 0) {
        sign = 1;
        break;
    } else if (val < 0) {
        sign = -1;
        break;
    }
}
if (sign <= 0) {
    faces.insert(faces.begin(), face);
} else {

```

```

        faces.emplace_back(face);
    }
}
return faces;
}

int main(){
    ios_base::sync_with_stdio(0);cin.tie(NULL);

    // Nos van a dar n aristas
    ll n; cin>>n;

    // Almacenamiento de vertices y aristas para llamar el
    algoritmo de findFaces
    vector<Point> vertices;
    vector<vector<ll>> adj;
    map<Point, ll> appear;
    for(ll i = 0; i < n; i++){
        ll x1, y1, x2, y2;
        cin>>x1>>y1>>x2>>y2;
        Point save1(x1, y1), save2(x2, y2);
        ll index1, index2;
        map<Point, ll>::iterator it1, it2;
        it1 = appear.find(save1);
        it2 = appear.find(save2);
        if(it1 == appear.end()) {
            index1 = appear[save1] = vertices.size();
            vertices.pb(save1);
            adj.emplace_back();
        }
        else

```

```
    index1 = it1->second;

    if(it2 == appear.end()) {
        index2 = appear[save2] = vertices.size();
        vertices.pb(save2);
        adj.emplace_back();
    }
    else
        index2 = it2->second;

    adj[index1].pb(index2);
    adj[index2].pb(index1);
}

vector<vector<ll>> faces = find_faces(vertices, adj);
return 0;
}
```

3. Flow

3.1 MaxDlow Dinic

```
struct comp{
    bool operator() (pii a, pii b){
        if(a.se == b.se)
            return a.fi < b.fi;
        return a.se < b.se;
    }
};

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vector<int> lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0){
        adj[a].pb({b, (int)adj[b].sz, c, c});
        adj[b].pb({a, (int)adj[a].sz - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f){
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < adj[v].sz; i++){
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))){
                    e.c -= p, adj[e.to][e.rev].c += p;
```

```
                    return p;
                }
            }
        }
        return 0;
    }
    ll calc(int s, int t){
        ll flow = 0; q[0] = s;
        for(ll L = 0; L < 31; L++) do { // 'int L=30' maybe faster for
random data
            lvl = ptr = vector<int> (q.sz);
            int qi = 0, qe = lvl[s] = 1;
            while(qi < qe && !lvl[t]){
                int v = q[qi++];
                for(Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qi++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; } // 1 si es parte del
lado de source
};

int main(){
    ios_base::sync_with_stdio(0);cin.tie(NULL);
    int n; cin>>n;
    Dinic graph(n + 2); //Crear un grafo para usar Dinic

    // Agregar una arista desde s hasta d con capacidad c
```

```

// Tener en cuenta que es un grafo dirigido, si se quiere un grafo
no dirigido,
// agregar la misma arista en direccion contraria
graph.addEdge(s, d, c);

graph.calc(source, sink); //Maximo flujo desde source hast sink

// Para reconstruir el flujo enviado por cuales aristas
for(int u = 0; u < N; ++u){
    for(auto &e : graph.adj[u]){
        int v = e.to;
        ll f = e.flow(); // f = flujo enviado desde u a v

        // Mostrar solo aristas "reales" (no las de retroceso)
        if(f > 0 && e.oc > 0){
            cout << "Flujo de " << u << " a " << v << ": " << f << endl;
        }
    }
}

// Para hallar los vertices que pertenecen al minimum vertex
cover
// Sea source = 0, el lado izquierdo del bipartito los nodos de 1
hasta n,
// el lado derecho los nodos de n+1 hasta 2*n y sink = 2*n + 1
for(auto &e : graph.adj[0]){ // Recorremos la lista de adyacencia
de source
    int v = e.to;
    if(!graph.leftOfMinCut(v)) cout<<1<<' '<<v<<endl; // Si v no
esta en left en el corte, hace parte del MVC
}
for(int u = n+1; u <= 2*n; u++){

```

```

        if(!graph.leftOfMinCut(u)) continue; // Recorremos los nodos
que hacen parte de left en el corte y en la derecha del bipartito
        for(auto &e : graph.adj[u]){
            int v = e.to;
            if(v != 2*n + 1) continue; // Si tiene arista hacia sink, hace
parte del MVC
            cout<<2<<' '<<u - n<<endl;
            break;
        }
    }

    return 0;
}

```

3.2 MinCost MaxFlow

```

/**
 * Author: Stanford
 * Date: Unknown
 * Source: Stanford Notebook
 * Description: Min-cost max-flow.
 * If costs can be negative, call setpi before maxflow, but note
that negative cost cycles are not supported.
 * To obtain the actual flow, look at positive values only.
 * Status: Tested on kattis:mincostmaxflow, stress-tested against
another implementation
 * Time:  $O(F \cdot E \cdot \log(V))$  where F is max flow.  $O(V \cdot E)$  for setpi.
 */
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
using ll = long long;
using ld = long double;

```



```

#define pb push_back
#define sz size()
#define fi first
#define se second
#define all(a) a.begin(), a.end()
typedef pair<int, int> pii;
// #include<ext/pb_ds/assoc_container.hpp>
// using namespace __gnu_pbds;
// using ordered_set = tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update>;

typedef pair<ll, ll> pll;
#include <ext/pb_ds/priority_queue.hpp>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vector<int> seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;

```

```

        ed[from].push_back(edge{ from, to, (int)ed[to].sz, cap, cost, 0
    });
        ed[to].push_back(edge{ to, from, (int)ed[from].sz-1, 0, -cost, 0
    });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.to });
                }
            }
        }
        for (int i = 0; i < N; i++) pi[i] = min(pi[i] + dist[i], INF);
    }
}

```

```

pll maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    for(int i = 0; i < N; i++) for(edge& e : ed[i]) totcost += e.cost *
e.flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        for(int i = 0; i < N; i++) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};

```

3.3 Matching Ponderado (Algoritmo Hungaro)

```

/*
    0-index.
    Sea n la cantidad en el lado izquierdo y m la cantidad en el lado
    derecho, se requiere  $n \geq m$ , es decir, deben haber igual o mas
    workers que jobs.
    Minimo costo para el maximum matching de un bipartito
    ponderado.
    Para explicar el algoritmo se documenta utilizando worker (W)
    como L
    y job (J) como R. Cada trabajador cobra cierta cantidad por
    realizar
    cierto trabajo sin embargo cada trabajador solo puede realizar
    un
    trabajo. Se halla el menor costo para realizar todos los
    trabajos.
    Complejidad:  $O(J * W^2)$ 
    Suele ser un poco mas eficiente que el algoritmo de MaxFlow
    MinCost.
*/

int ckmin(int &a, int b) { return a > b ? ((a = b), true) : false; }

/**
 * return the jobs of each worker in the optimal assignment,
 * or -1 if the worker is not assigned
 */
template <class T> vector<int> hungarian(const
vector<vector<T>> &C){
    int J = C.size();
    int W = C[0].size();

```

```
assert(J <= W); // Verifica que el lado izquierdo (W) sea mayor o
igual al lado derecho (J)
```

```
// job[w] = job assigned to w-th worker, or -1 if no job assigned
// note: a W-th worker was added for convenience
vector<int> job(W + 1, -1);
vector<T> h(W); // Johnson potentials
```

```
const T inf = numeric_limits<T>::max();
// assign j_cur-th job using Dijkstra with potentials
for(int j_cur = 0; j_cur < J; j_cur++){
    int w_cur = W; // unvisited worker with minimum distance
    job[w_cur] = j_cur;

    vector<T> dist(W + 1, inf); // Johnson-reduced distances
    dist[W] = 0;
    vector<bool> vis(W + 1); // whether visited yet
    vector<int> prv(W + 1, -1); // previous worker on shortest path
    while(job[w_cur] != -1){ // Dijkstra step: pop min worker from
```

```
heap
    T min_dist = inf;
    vis[w_cur] = true;
    int w_next = -1; // next unvisited worker with minimum
distance
```

```
    // consider extending shortest path by w_cur -> job[w_cur] -
> w
    for(int w = 0; w < W; w++){
        if(!vis[w]){
            // sum of reduced edge weights w_cur -> job[w_cur] ->
w
            T edge = C[job[w_cur]][w] - h[w];
```

```
        if (w_cur != W) {
            edge -= C[job[w_cur]][w_cur] - h[w_cur];
            assert(edge >= 0);
        }
        if (ckmin(dist[w], dist[w_cur] + edge)) { prv[w] = w_cur; }
        if (ckmin(min_dist, dist[w])) { w_next = w; }
    }
}
w_cur = w_next;
}
```

```
for(int w = 0; w < W; w++){ // update potentials
    ckmin(dist[w], dist[w_cur]);
    h[w] += dist[w];
}
```

```
while(w_cur != W){ // update job assignment
    job[w_cur] = job[prv[w_cur]];
    w_cur = prv[w_cur];
}
}
```

```
return job; // Retorna la mejor asignacion encontrada
}
```

```
int main(){
    int n;
    cin >> n;
    vector<vector<int>> c(n, vector<int>(n));
```

```
    for(int i = 0; i < n; i++){ // ith job
        for(int j = 0; j < n; j++){ // jth worker
```

```
        cin>>c[i][j];  
    }  
}
```

```
vector<int> mat = hungarian(c);
```

```
    // Se suma segun el costo de que el job mat[i] sea realizado por  
    el worker i  
    int cost = 0;  
    for (int i = 0; i < n; i++) cost += c[mat[i]][i];  
    cout<<cost<<endl;  
    for(int i = 0; i < n; i++) cout<<mat[i]<<' '<<i<<endl; // El job mat[i]  
    es realizado por el worker i  
}
```

4. Range Queries

4.1 Sparse Table (Maximos)

const int lgn = 17; //Piso del Lg2 de MAXN (maximo numero de elementos)

const ll maxn = 2e5 + 2;

typedef pair<ll, int> pli;

/*

Complejidad:

- Queries $O(1)$
- Creacion $O(n * \lg(n))$

*/

/*

- Si bien en este ejemplo se usa 1-index, el algoritmo es valido para 0-index, simplemente se deben cambiar los limites de i dentro del for(i = 1; i <= n; i++) por for(i = 0; i < n; i++) tanto en la lectura de los datos como en el llenado de la sparse table.

- En esta implementacion tambien se calcula el indice en el cual se encuentra el elemento mas pequeño, se puede eliminar simplemente dejando el primer elemento del par

- Para valor minimo simplemente cambiar los max por min

*/

vector<ll> a(maxn);

pli rmq[maxn][lgn + 1];

void solver(){

int n;cin>>n;

for(int i = 1; i <= n; i++){

cin>>a[i];

rmq[i][0] = {a[i], i}; //Inicializar sparse table

}

//Llenar sparse table

for(int j = 1; j <= lgn; j++){

for(int i = 1; i + (1 << j) - 1 <= n; i++){

rmq[i][j] = max(rmq[i][j - 1], rmq[i + (1 << (j - 1))][j - 1]);

}

//Queries

int q; cin>>q;

while(q--){

int l, r; cin>>l>>r;

int lg = 31 - __builtin_clz(r - l + 1); // Piso del Log2 del tamaño del rango

pli maxi = max(rmq[l][lg], rmq[r - (1 << lg) + 1][lg]); // [l, r]

cout<<"El mayor numero entre "<<l<<" y "<<r<<" es:

"<<maxi.first<<", y se encuentra en la posicion:

"<<maxi.second<<endl;

}

}

4.2 Segment Tree (Segmento con mayor suma)

struct Node{

ll seg, pref, suf, sum; // El resultado del segmento con mayor suma se guarda en seg

};

```

const int maxn = 1e5 + 1;
const ll ninf = -1e18;
vector<Node> tree(4*maxn);

Node op(Node a, Node b){
    Node ans;
    ans.seg = max(0LL, max(max(a.seg, b.seg), a.suf + b.pref));
    ans.pref = max(0LL, max(a.pref, a.sum + b.pref));
    ans.suf = max(0LL, max(b.suf, b.sum + a.suf));
    ans.sum = a.sum + b.sum;

    return ans;
}

void build(vector<ll>& a, int v, int tl, int tr){
    if(tl == tr) tree[v] = {max(0LL, a[tl]), max(0LL, a[tl]), max(0LL,
a[tl]), a[tl]};
    else{
        int tm = (tl + tr) / 2;
        build(a, 2*v, tl, tm);
        build(a, 2*v + 1, tm + 1, tr);
        tree[v] = op(tree[2*v], tree[2*v + 1]);
    }
}

// Las queries son [l, r]
Node query(int v, int tl, int tr, int l, int r){
    if(l > r) return {0, 0, 0, ninf};
    if(tl == l && tr == r) return tree[v];

    int tm = (tl + tr) / 2;
    Node a = query(2*v, tl, tm, l, min(tm, r));

```

```

    Node b = query(2*v + 1, tm + 1, tr, max(tm + 1, l), r);

    return op(a, b);
}

void update(int v, int tl, int tr, int pos, ll val){
    if(tl == tr) tree[v] = {max(0LL, val), max(0LL, val), max(0LL,
val), val};
    else{
        int tm = (tl + tr) / 2;
        if(pos <= tm) update(2*v, tl, tm, pos, val);
        else update(2*v + 1, tm + 1, tr, pos, val);
        tree[v] = op(tree[2*v], tree[2*v + 1]);
    }
}

```

4.3 Lazy Propagation

// Segment Tree Lazy Propagation

// - Es necesario que se cumpla la propiedad asociativa tanto en las operaciones de update como calc.

// - Es necesario que se cumpla que $\text{calcOp}(\text{updateOp}(a, x), \text{updateOp}(b, x)) == \text{updateOp}(\text{calcOp}(a, b), x)$,

// es decir, que updateOp sea distributiva relativa a calcOp , ejemplo, update de multiplicacion, calc

// suma. En caso de que no se cumpla esa propiedad (update assign , calc suma o update suma , calc suma),

// se debe ajustar utilizando la longitud del rango en la operacion update.

// - Este ejemplo corresponde a update assign y calc suma .

```

class segTree {
private:

```

```

int size;
vector<ll> lazy;
vector<ll> tree;

ll neutro = LLONG_MAX - 1;

ll updateOp(ll a, ll b, ll len){
    if(b == neutro) return a;
    return b * len;
}

ll calcOp(ll a, ll b){
    if(a == neutro) return b;
    if(b == neutro) return a;
    return a + b;
}

void applyUpdOp(ll &a, ll b, ll len){
    a = updateOp(a, b, len);
}

void propagate(int v, int tl, int tr){ // O(1)
    if(tr - tl == 1) return;
    int tm = (tr + tl) / 2;
    applyUpdOp(lazy[2*v + 1], lazy[v], 1);
    applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
    applyUpdOp(lazy[2*v + 2], lazy[v], 1);
    applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
    lazy[v] = neutro;
}

void update(int l, int r, ll val, int v, int tl, int tr){ // O(lg(n))

```

```

    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return;
    if(tl >= l && tr <= r){
        applyUpdOp(lazy[v], val, 1);
        applyUpdOp(tree[v], val, tr - tl);
        return;
    }

    int tm = (tl + tr) / 2;
    update(l, r, val, 2*v + 1, tl, tm);
    update(l, r, val, 2*v + 2, tm, tr);
    tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
}

ll calc(int l, int r, int v, int tl, int tr){ // O(lg(n))
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return neutro;
    if(tl >= l && tr <= r) return tree[v];

    int tm = (tl + tr) / 2;
    ll m1 = calc(l, r, 2*v + 1, tl, tm);
    ll m2 = calc(l, r, 2*v + 2, tm, tr);
    return calcOp(m1, m2);
}

// void build(int v, int tl, int tr){ // O(n)
//     if(tr == tl + 1){
//         tree[v] = 1;
//         return;
//     }
//     int tm = (tr + tl) / 2;
//     build(2*v + 1, tl, tm);

```

```

// build(2*v + 2, tm, tr);
// tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
// }

```

public:

```

void init(int n){
    size = 1;
    while(size < n) size *= 2;
    lazy.assign(2*size, 0LL);
    tree.assign(2*size, 0LL);
    // build(0, 0, size);
}

```

```

void update(int l, int r, ll val){
    update(l, r, val, 0, 0, size);
}

```

```

ll calc(int l, int r){
    return calc(l, r, 0, 0, size);
}
};

```

```

void solver(){
    int n, m; cin>>n>>m;
    segTree st;
    st.init(n);
    while(m--){
        int op; cin>>op;
        if(op == 1){
            int l, r; ll val; cin>>l>>r>>val;
            st.update(l, r, val);
        }
    }
}

```

```

else{
    int l, r; cin>>l>>r;
    cout<<st.calc(l, r)<<endl;
}
}
}
}

```


5. DP

5.1 LIS

5.1.1 Binary Search

```
const int inf = 1e9 + 1;
```

```
// Complejidad: O(n * log(n))
// - En este problema me piden el tamaño de la Longest
Increasing
// Subsequence (LIS). Una LIS es un array a donde para cada
// i, j con i < j, a[i] < a[j].
// - Para hallar una Longest Non-Decreasing Subsequence, es
decir,
// el tamaño del mayor array a donde para cada i < j, a[i] <= a[j],
// solo hay que revisar la inecuación de la búsqueda binaria
// - La Longest Decreasing Subsequence (LDS) es equivalente a el
LIS
// de un array invertido
// - Si me piden el menor número de Increasing Subsequences,
eso es
// equivalente al tamaño de la Longest Non-Increasing
Subsequence.
// Es decir, el mayor tamaño de array a, donde para cada
// i < j, a[i] >= a[j]
// - Si me piden la mayor subsecuencia de números
consecutivos, se
// se puede usar un enfoque con sets o mapas, en el que se
guarde
// cada valor en donde termina una subsecuencia consecutiva,
y el
// tamaño de esta. Entonces cada que queramos agregar un
valor x,
```

```
// buscamos si existe una subsecuencia que termine en el x - 1,
// con longitud l, en caso de que si la eliminamos y agregamos
una
// nueva subsecuencia que termina en x con longitud l + 1.
Finalmente,
// recorreremos todas las subsecuencias guardadas en el set o
map y nos
// quedamos con la que tenga mayor longitud. Para reconstruir
los
// índices podemos encontrar la última ubicación del número
mayor x de
// subsecuencia más larga y recorrer hacia la izquierda
buscando x-1,
// después x-2, hasta llegar a x - l + 1.
```

```
void solver(){
    int n; cin >> n;
    vector<int> a(n+1);
    for(int i = 1; i <= n; i++) cin >> a[i];

    vector<int> d(n+1, inf); d[0] = -inf;

    for(int i = 1; i <= n; i++){
        vector<int>::iterator it = upper_bound(d.begin(), d.end(), a[i]);
        if(it == d.end()) continue;

        int index = distance(d.begin(), it);
        // Este if se encarga de no poner números iguales
        // Si me piden que cada número sea estrictamente mayor
        // Si piden mayor o igual se puede quitar
        if(a[i] > d[index - 1]) d[index] = a[i];
    }
```

```

int ans = 0;
for(int i = 1; i <= n; i++){
    if(d[i] < inf) ans = i;
}

cout<<ans<<endl;
}

```

5.1.2 Segment Tree

```
const int maxn = 200001, ninf = -1e9 - 5;
```

```
// Esta implementacion es 0-index
/*
```

Para hallar el LIS usando Segment Tree, decimos que $t[a[i]] = \max(t[0], t[1], \dots, t[a[i] - 1]) + 1$. Esta definicion quiere decir que el LIS que puedo obtener finalizando en el valor $a[i]$ es igual a 1 mas el mayor de los LIS para $j < i$ && $a[j] < a[i]$. Para hallar el valor mayor usamos Segment Tree.

Adicionalmente, para saber la cantidad total de Increasing Subsequence's que tiene el array a , simplemente tenemos que modificar este enfoque haciendo que el Segment Tree no nos de el maximo en un rango si no la suma en un rango. Tener en cuenta que el valor puede ser muy grande asi que es necesario usar modularidad.

```
*/
```

```
// Complejidad:  $O(n * \lg n)$ 
// El arbol del segment tree va a servir para representar el array t
vector<int> tree(4*maxn);
int n;
```

```
void updateTree(int update, int value){
    tree[n + update] = value;
    update += n;
    for(int i = update; i > 1; i >= 1) tree[i >= 1] = max(tree[i], tree[i ^ 1]); //Si i es el hijo por izquierda de i/2, entonces i ^ 1 es el hijo por derecha, y viceversa
    //i >= i -> i/2
}
```

```
// La implementacion es [l, r)
int query(int l, int r){
    int ans = ninf;
    l += n; r += n;
    while(l < r){
        if(l & 1) ans = max(ans, tree[l++]);
        if(r & 1) ans = max(ans, tree[--r]);
        l >= 1; r >= 1;
    }

    return ans;
}
```

5.2 Digit DP

```
/*
Digit DP se utiliza en ejercicios en los que se nos pide contar la cantidad de numeros en un rango

```

muy grande (entre 1 y 10^{18}) cuyos digitos cumplen determinadas condiciones. Lo que se hace es un dp a traves de los digitos de cada numero, con una dimension para saber en que digito vamos, otra dimension para guardar si ya nos aseguramos que el numero sea menor al mayor numero en rango y mas dimensiones dependiendo de las condiciones del ejercicio. Se calcula la respuesta en rango ansL = (0, L - 1] y ansR = (0, R] y la respuesta definitiva es ansR - ansL.

En este ejemplo se quiere calcular la suma de los digitos para cada numero entre L y R, asi que la dimension extra es la suma.

*/

// La tercera dimension representa la suma de los digitos, como el numero mayor es 10^{18} , sabemos que // a lo sumo la suma de digitos de algun numero sera $19 * 9 = 171$, asi que se asigna una talla donde

// entre ese valor.

ll dp[20][2][180];

vector<int> num;

int len;

// Se convierte el numero en vector para hacer facilmente comparaciones entre digitos

void intToVec(ll x){

num.clear();

len = 0;

for(int i = 0; i < 20; i++){

num.pb(x % 10);

len++;

x /= 10;

}

reverse(num.begin(), num.end());

}

// Se limpia el dp para calcular el resultado para el siguiente numero

void reset(){

for(int i = 0; i < 20; i++){

for(int j = 0; j < 2; j++){

for(int k = 0; k < 180; k++){

dp[i][j][k] = -1;

}

}

}

}

ll calc(int idx, bool under, int sum){

if(idx == len) return sum; // Si se llega a la longitud maxima, se retorna la suma

if(dp[idx][under][sum] != -1) return dp[idx][under][sum];

// Si ya nos aseguramos que estamos viendo un numero menor a R, el siguiente digito

// agregado puede ser cualquiera, es decir, puede ir de 0 a 9; de lo contrario, debemos

// asegurarnos que el siguiente digito agregado sea menor o igual al digito de R, pues

// de lo contrario ya estariamos probando un numero mayor.

```

int lim = under ? 9 : num[idx];

ll ans = 0; // Se inicializa la respuesta en 0

for(int i = 0; i <= lim; i++){
    // Si ya sabemos que estamos analizando numeros menores,
    // o el digito que vamos a agregar es
    // estrictamente menor al de R, haremos el siguiente llamado
    // sabiendo que estamos analizando
    // numeros menores
    bool nUnder = under || i < num[idx];

    // Se llama con el siguiente indice de digito, aclarando si ya
    // nos aseguramos que los numeros
    // a analizar son menores a R, y a la suma se le agrega el
    // digito que se va a poner
    ans += calc(idx + 1, nUnder, sum + i);
}

dp[idx][under][sum] = ans;
return ans;
}

ll solver(ll x){
    if(x <= 0) return 0;

    intToVec(x);
    reset();
    return calc(0, 0, 0);
}

int main(){

```

```

ios_base::sync_with_stdio(0);cin.tie(NULL);

```

```

int t;
cin>>t;
while(t--){
    ll l, r; cin>>l>>r;
    cout<<solver(r) - solver(l-1)<<endl;
}

return 0;
}

```

5.3 Bitmask DP

5.3.1 TSP

/*

- Implementacion 0-index
- Complejidad $O(2^n * n^2)$
- Halla el minimo peso para recorrer todos los vertices del grafo iniciando en el vertice 0 (teniendo en cuenta que para moverse entre 2 vertices solo se pueden usar aristas entre ellos).
- Otras aplicaciones:
 - Para hallar el camino hamiltoniano de menos costo (empezando en cualquier vertice) simplemente se inicializa el DP con $dp[i][1 < i] = 0$, para i entre 0 y $n-1$.
 - Para pedir que termine en algun vertice en especifico, por ejemplo $n-1$, simplemente hacer que la unica mascara valida que contenga $n-1$ sea $lim-1$, y que solo sea valida cuando el vertice final sea $n-1$, es decir,

```

    dp[n-1][lim-1].
- Para hallar el ciclo hamiltoniano mas corto, podemos elegir el
vertice
    0 como inicial y marcar dp[0][1] = 0 e inicializar los demas
estados en
    inf, despues corremos el algoritmo con normalidad y
finalmente para
    cada  $1 \leq i < n$  calculamos  $dp[i][lim-1] + dis(0, i)$  y elegimos el
menor.
*/

const int maxn = 20, inf = 1e9;
vector<vector<pii>> graph(maxn);

void solver(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++){
        int u, v, w; cin >> u >> v >> w; u--; v--;
        graph[u].pb({v, w});
        graph[v].pb({u, w});
    }

    // Inicializar el dp
    int lim = 1 << n;
    vector<vector<int>> dp(n, vector<int>(lim, inf));
    dp[0][1] = 0;

    for(int mask = 1; mask < lim; mask++){
        if(mask & 1 == 0) continue; // Si no he visitado el vertice
inicial
        for(int u = 0; u < n; u++){

```

```

            if(mask && (1 << u) == 0) continue; // Si no he visitado el
vertice donde quiero finalizar

            for(pii e : graph[u]){
                int v = e.fi, w = e.se;
                if(mask & (1 << v) == 0) continue; // Si no he visitado el
vertice desde el cual vendria
                dp[u][mask] = min(dp[u][mask], w + dp[v][mask ^ (1 <<
u)]);
            }
        }
    }

    int ans = inf;
    for(int i = 1; i < n; i++) ans = min(ans, dp[i][lim - 1]);
}

```

5.3.2 Counting Cycles

```

const int maxn = 20;
vector<vector<int>> graph(maxn);

// Complejidad  $O(2^n * n^2)$ 
void solver(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++){
        int u, v; cin >> u >> v; u--; v--;
        graph[u].pb(v);
        graph[v].pb(u);
    }

    int lim = (1 << n);
    vector<vector<ll>> dp(n, vector<ll>(lim));

```

```

// Contamos los "simple path" que contiene el grafo que
// empiezan por el primer bit encendido de la mascara y
// terminan en i

// Cada camino de 1 vertice se puede hacer de una forma
for(int i = 0; i < n; i++) dp[i][1 << i] = 1;

// Si i no es el primer vertice activado y hay arista j -> i
// sumamos lo que hubiera en el subconjunto mask, quitando a
i
// y que terminaba en j
for(int mask = 3; mask < lim; mask++){
    for(int i = 0; i < n; i++){
        if(!(mask & (1 << i)) || !(mask % (1 << i))) continue;

        for(int j : graph[i]){
            if(!(mask & (1 << j))) continue;
            dp[i][mask] += dp[j][mask ^ (1 << i)];
        }
    }
}

// Para cada mask cuyo primer vertice sea fst y ultimo vertice i
// sumamos dp[mask][i] a la respuesta si existe arista i -> fst
// ya que esta es la que cerraria el ciclo
ll ans = 0;
for(int mask = 7; mask < lim; mask++){
    for(int i = 0; i < n; i++){
        int count = 0, fst = n;
        for(int j = 0; j < n; j++){
            if(mask & (1 << j)){

```

```

                count++;
                fst = min(fst, j);
            }
        }
        if(count < 3 || fst == i) continue;

        bool cond = 0;
        for(int v : graph[i]){
            if(v == fst) cond = 1;
        }
        if(cond) ans += dp[i][mask];
    }
}
ans /= 2;
// Dividimos la respuesta por 2 ya que los ciclos estan siendo
considerados en ambas direcciones
cout<<ans<<endl;
}

5.3.3 Counting Hamiltonian Paths
const int mod = 1e9 + 7, maxn = 20;
int dp[maxn][1 << maxn];
vector<vector<int>> graph(20);

// - Cuenta la cantidad de caminos hamiltonianos que contiene
un grafo
// empezando en el vertice 1, pasando por todos los vertices
// exactamente una vez y terminando en el vertice n
// - Para contar la cantidad de caminos hamiltonianos que
contiene un
// grafo iniciando y terminando en cualquier vertice,
inicializamos

```

```

// dp[i][1 << i] en 1, para i desde 1 hasta n y finalmente sumamos
// dp[i][2^n - 1] para i desde 1 hasta n
// - Para contar la cantidad de caminos simples que contiene un
grafo
// (que pasa por algunos vertices exactamente una vez)
hacemos el
// mismo dp que en el caso anterior pero esta vez sumamos
cada
// estado cuyo subconjunto contenga 2 o mas vertices y
finalmente
// dividimos la respuesta entre 2 ya que de esa forma se esta
// contando 2 veces cada camino (en ambas direcciones)
// - Para contar la cantidad de ciclos hamiltonianos que tiene un
grafo
// podemos marcar algun nodo en especifico como inicial, por
ejemplo
// el 0 y contar todos los caminos hamiltonianos que empiezan
en ese
// nodo. Finalmente, para cada 1 <= i < n, si hay una arista que
// conecte i con 0, sumamos dp[i][lim-1].

// Complejidad: O(2^n * n^2)
void solver(){
    int n, m; cin>>n>>m;
    for(int i = 0; i < m; i++){
        int u, v; cin>>u>>v;
        u--; v--;
        graph[u].pb(v);
    }

    int lim = 1 << n;
    dp[0][1] = 1;

```

```

for(int j = 1; j < lim; j++){

    // Estos 2 ifs eliminan muchas posibilidades, así que pueden
ayudar a evitar TLE
    if(!(j & 1)) continue; // Si no he pasado por la fuente (1 << s)
    if(j & (1 << (n-1)) && j < lim - 1) continue; // Si llego al destino
sin haber pasado por los demas (1 << d)

    for(int i = 0; i < n; i++){

        if(!(j & (1 << i))) continue;

        for(int v : graph[i]){
            if(j & (1 << v)) continue;
            dp[v][j | (1 << v)] += dp[i][j];
            dp[v][j | (1 << v)] %= mod;
        }
    }
}

cout<<dp[n-1][lim - 1]<<endl;
}

```

5.3.4 Recorrer Subconjuntos

```

// - Complejidad O(3^n)
// Si contamos el numero de pares ordenados (T, S), donde T
esta contenido
// en S, tenemos que cada elemento x tiene 3 opciones: esta en
T y en S,
// esta solo en S, no esta en ninguno. Pues si x estuviera en T y
no en S,

```

```
// entonces T no estaria contenido en S. Por lo tanto, cada
elemento puede
// estar en 3 posibles estados.
// - Cuando se le resta uno a la submascara, el bit en 1 mas a la
derecha
// pasa a ser 0 y todos los bits a su derecha se convierten en 1.
Luego se
// aplica AND para eliminar todos los bits que no estan en la
mascara.
// Finalmente se usa la XOR para recorrer los subconjuntos en
orden
// incremental
// - Source: https://usaco.guide/gold/dp-bitmasks
for(int mask = 0; mask < (1 << n); mask++){
    for(int submask = mask; submask != 0; submask = (submask -
1) & mask){
        int subset = mask ^ submask;
        // do whatever you need to do here
        // submask contiene todo lo que esta en mask pero no en
subset
    }
}
```


6. Otros

6.1 Area de Un Poligono

// Estructura Point con campos "x" y "y" y operaciones entre puntos

```
struct Point {
```

```
    ll x, y;
```

```
    Point(ll x_, ll y_): x(x_), y(y_) {}
```

```
    bool operator < (const Point& p) const {
```

```
        return tie(x, y) < tie(p.x, p.y);
```

```
    }
```

```
    Point operator - (const Point& p) const {
```

```
        return Point(x - p.x, y - p.y);
```

```
    }
```

```
    ll cross (const Point & p) const {
```

```
        return x * p.y - y * p.x;
```

```
    }
```

```
    ll cross (const Point & p, const Point & q) const {
```

```
        return (p - *this).cross(q - *this);
```

```
    }
```

```
    ll half () const {
```

```
        return ll(y < 0 || (y == 0 && x < 0));
```

```
    }
```

```
};
```

//P es un polígono ordenado anticlockwise.

//Si es clockwise, retorna el area negativa.

```
//P[0] != P[n-1]
```

```
ld PolygonArea(const vector<Point> &p){
```

```
    ld r = 0.0;
```

```
    for (ll i=0; i<p.size(); ++i){
```

```
        ll j = (i+1) % p.size();
```

```
        r += p[i].x*p[j].y - p[j].x*p[i].y;
```

```
    }
```

```
    return r/2.0;
```

```
}
```

6.2 Hashing

// Algunas notas:

// - Usar Hashing en codeforces puede ser peligroso debido al hacking

// - La probabilidad de que 2 cadenas tengan el mismo hashing es de n / m

// siendo n la longitud de la cadena mas larga

// - Si tengo un conjunto de hashes de k cadenas con longitud de maximo n,

// la probabilidad de que 2 cadenas tengan igual hash es de $(k^2 * n) / m$

// Por ende, si k y n son de alrededor de $1e5$ y m alrededor de $1e9$, es

// casi seguro que habra colisiones. Una forma de solucionarlo puede ser

// un primo m mas largo, como $2^{61} - 1$

(2305843009213693951) y usar

// enteros de 128 bits para las multiplicaciones, aunque ahora el hashing

// y las queries serian un poco mas lentas. (Para hacerlo puedo simplemente

// poner using ll = __int128; y cambiar el modulo).

```
// Halla el hashing de una cadena en O(n)
ll compute_hash(string s){
    ll p = 31; // Si s se compone de lowercase english letters
    // ll p = 53; // Si s se compone de lowercase y uppercase
    english letters
    ll p_pow = 1; // Si precalculo las potencias de p podria ser
    ligeramente mas eficiente
    ll m = 1e9 + 9;
    ll ans = 0;

    for(char c : s){
        ans = (ans + (c - 'a' + 1) * p_pow) % m;
        p_pow = (p_pow * p) % m;
    }

    return ans;
}

// Sirve para hacer precalculos para hacer queries de la
subcadena
// sl, sl+1, ..., sr
// Construir el array toma O(n), las queires son O(1)
void precalc_hashing(string s, vector<ll>& h, vector<ll>& ph){
    ll p = 31;
    ll m = 1e9 + 9;

    h[0] = s[0] - 'a' + 1;
    ph[0] = 1;

    for(int i = 1; i < s.sz; i++){
        h[i] = (((h[i-1] * p) % m) + (s[i] - 'a' + 1)) % m;
```

```
        ph[i] = (ph[i-1] * p) % m;
    }

    /*
    Para hacer queries:
    - Si l = 0: h[r]
    - Si no: (h[r] - h[l-1] * ph[r-l+1]) % m
    */
}

ll queries(ll l, ll r, vector<ll>& h, vector<ll>& ph){
    ll m = 1000000009;
    if(!l) return h[r];
    return (h[r] - ((h[l-1] * ph[r-l+1]) % m) + m) % m;
}
```

6.3 Ternary Search (No la he testeado)

```
int ternarySearch(vector<int>& A, int i, int j, int s) {
    int l, r;
    while(i <= j) {
        l = i + (j-i)/3, r = j - (j-i)/3;
        if(A[l] == s)
            return l;
        if(A[r] == s)
            return r;
        if(A[l] > s)
            i = r+1;
        else if(A[r] > s) {
            i = l+1;
            j = r-1;
        }
        else
```

```

        j = l-1;
    }
    return -i -1;
}

```

6.4 Métodos deque

```

deque<int> dq;
dq.push_front(1);
dq.push_back(10);
deque<int>::iterator it = dq.begin();
it++;
dq.insert(it, 5); // O(n)
it = dq.end();

```

```

int n = dq.front(); //1
int m = dq.back(); //10
int x = dq.at(1); //5

```

```

dq.pop_front();
dq.pop_back();
it = dq.begin();
dq.erase(it); // O(n)

```

```

int len = dq.size();

```

```

dq.clear();

```

6.5 Métodos listas

/* A partir de estas funciones se pueden simular listas
 ascendentes,
 circulares, enlazadas y doblemente enlazadas */

```

list<int>::iterator next(list<int> &lista, list<int>::iterator i) {
    i++;
    if(i == lista.end())
        i = lista.begin();
    return i;
}

```

```

list<int>::iterator prev(list<int> &lista, list<int>::iterator i) {
    if (i == lista.begin())
        i = lista.end();
    i--;
    return i;
}

```

```

int main() {
    ios_base::sync_with_stdio(0);cin.tie(NULL);
    list<int> lista;
    lista.push_back(1);
    int len = lista.size();
    list<int>::iterator it1 = lista.begin();
    list<int>::iterator it2 = lista.end();
    lista.erase(it1);
    list<int>::iterator s1 = next(lista, s1);
    list<int>::iterator s2 = prev(lista, s1);

    return 0;
}

```

6.6 Metodos indexed_set

// Set indexado
 // Sin elementos repetidos

// Se puedes simular elementos repetidos usando pii

```
#include<ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using indexed_set = tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update>;

indexed_set nombre; // Declaracion
nombre.insert(valor); // Insertar
nombre.size() // Numero de elementos
nombre.find_by_order(3) // Iterador al 4to elemento mas
pequeño
nombre.order_of_key(6) // Cuenta la cantidad de elementos
menores a 6
nombre.erase(valor) // Si lo encuentra lo elimina, sino no hace
nada
auto it = nombre.find(valor) // Devuelve un iterador a ese
elemento. Si no esta, devuelve un iterador al final
nombre.count(valor) // Retorna un entero, 1 si valor esta, 0 si no
esta
nombre.empty()
nombre.clear()
auto it = nombre.begin()
auto it = nombre.end()
auto it = nombre.lower_bound(5) // Retorna un iterador al primer
elemento mayor o igual a 5
auto it = nombre.upper_bound(5) // Retorna un iterador al primer
elemento mayor a 5
```

6.7 Para acceder a los elementos de un mapa

// Como acceder a los elementos de un mapa
map<int, int> T;

```
// Recorrer por valor o referencia
T[key]; // Accede al valor de key
for(const auto& par : T){
    key = par.first;
    valor = par.second;
}
```

```
// Por puntero o iterador
auto it = T.find(key);
if(it != T.end()){
    key = it->first;
    valor = it->second;
}
```

```
// lower_bound y upper_bound en set y map
auto it = T.lower_bound(key);
auto it = T.upper_bound(key);
```

6.8 Para cambiar el orden de contenedores

// En este caso se ordena revisando cual de los second es menor
y si son iguales

// se revisa cual de los first es menor

```
struct comparator {
    bool operator () (pii a, pii b){
        if(a.se == b.se) return a.fi < b.fi;
        return a.se < b.se;
    }
}
```

// Usar en:

```
sort(vec.begin(), vec.end(), comparator());
```

```
priority_queue<pii, vector<pii>, comparator> pq;  
set<pii, comparator> mySet;
```

6.9 Enteros de 128 bits

```
// Tomado del template de Jiangly  
using u128 = unsigned __int128;  
using i128 = __int128;  
  
// - Solo lo soportan algunos compiladores  
// - Solo disponible en arquitecturas de 64 bits (como x86_64)  
// - MSVC (Microsoft Visual C++) no lo soporta directamente  
// - En CF se puede desde C++ 20 (la 17 es arquitectura de 32  
bits)  
  
// Convertir en cadena para imprimir  
i128 x;  
string s = "";  
while(x > 0){  
    s += '0' + (x % 10);  
    x /= 10;  
}  
reverse(s.begin(), s.end());
```

6.10 Imprimir Decimales

```
cout<<fixed<<setprecision(cantidad_decimales)<<ans<<endl;
```

7. Python

Es importante tener en cuenta que python es aproximadamente 40 veces mas lento que C++

```
import array as arr # Para trabajar con arrays
import numpy as np
```

```
def main ():
    # Leer una linea completa, separar los elementos por los
    espacios y almacenarlos en una lista
    read = input()
    numbers = read.split()
    numbers = list(map(int, numbers)) # 'map' si permite
    repeticiones, en cambio 'set' no
```

```
#Leer un entero
n = int(input())
```

```
# Leer un número flotante
d = float(input())
```

```
# Leer una cadena
s = input()
```

```
# Tamaño de una cadena
x = len(s)
```

```
# Agregar un caracter a una cadena
s2 += s1[i]
s2 += 'a'
```

```
# Negar un bool
```

```
cond = True
not cond # Esto seria False
```

```
# Potencia de un numero
ans = base ** exp
```

```
# Ciclo for
for i in range (0, 5): # Esto iria hasta el 4
for auto in autos: # autos seria una lista
```

```
# Condicionales
if(cond1):
elif(cond2):
else:
```

```
# Minimo y maximo
mina = min(mina, number)
maxa = max(maxa, number)
```

```
# Convertir un numero base x (guardado como cadena) en un
entero base 10
n10 = int(n2, x)
```

```
# Pasar un numero de base 10 a base 2 (se convierte a cadena)
ans2 = bin(ans10)[2:] # Para cambiar de base 10 a cualquier
otra base (que no sea base 2) toca a mano
```

```
# Evaluar una string que es una expresion matematica
(tambien puede tener parentesis)
result = eval(expresion)
```

```
# Metodos de listas (list)
```

```

# Resaltar que una sola lista puede contener varios tipos de
datos, incluida otra lista
lista = [1, "dos", [3, "cuatro"], 5.0, True] # Declarar
n = lista[i] # Sirve para acceder al elemento i de una lista
lista.append(6) # Agregar un elemento
lista.remove('dos') # Eliminar un elemento
lista.sort() # Ordenar
print(lista) # Imprimir toda la lista
lista.extend([7, 8]) # Agrega 7 y 8 a la lista
lista.insert(1, 'a') # Inserto 'a' en la posicion 1, el resto de
elementos corren un espacio a la derecha
n = lista.pop() # Elimina el ultimo elemento de la lista y se lo
asigna a 'n'
n = lista.pop(3) # Elimina el ultimo en la posicion 3 de la lista y
se lo asigna a 'n'
lista.clear() # Elimina todos los elementos la lista, quedando:
lista = []
lista.reverse # Invierte los elementos de la lista
indice = lista.index("cuatro") # Devuelve el indice de la primera
aparicion de "cuatro"
indice2 = lista.index(x, start) # Comienza a buscar desde el
indice start
indice3 = lista.index(x, start, end) # Devuelve la primera
aparicion de x en el rango [start, end)
conteo = lista.count(x) # Devuelve el numero de veces que
aparece 'x' en la lista

# Metodos de sets
# No tiene elementos repetidos, no esta ordenado
s = {1, 2, 3} # Declarar
s.add(4) # Agregar un elemento

```

```

s.remove(4) # Eliminar un elemento. Si el elemento no esta
lanza un error
s.discard(4) # Tambien elimina un elemento pero si no esta no
genera error
x = s.pop() # Elimina y retorna un elemento arbitrario
s.clear() # Vacía el conjunto
s1 = {1, 2}
s2 = {2, 3}
s3 = {4, 5, 6}

# Metodos de operaciones de conjuntos
# Union de conjuntos
s4 = s1.union(s2, s3)
s4 = s1 | s2 | s3

# Interseccion de conjuntos
s4 = s1.intersection(s2, s3)
s4 = s1 & s2 & s3

# Retorna lo que esta en el primer conjunto pero no en los
demas
s4 = s1.difference(s2, s3)
s4 = s1 - s2 - s3

# Los elementos que solo estan en un conjunto
# Se recomienda usarla solo para dos conjuntos
s4 = s1.symmetric_difference(s2).symmetric_difference(s3)
s4 = (s1 ^ s2) ^ s3

# Metodos para relaciones de conjuntos

```

```
s1.issubset(s2) # Devuelve True si s1 es subconjunto de s2
s1.issuperset(s2) # Devuelve True si s1 es superconjunto de s2
s1.isdisjoint(s2) # Devuelve True si no tienen elementos en
comun
```

Metodos de actualizacion

```
# Actualiza s1 con la union entre s1 y s2
s1.update(s2)
s1 |= s2
```

```
# Actualiza s1 con la interseccion entre s1 y s2
s1.intersection_update(s2)
s1 &= s2
```

```
# Actualiza s1 eliminando los elementos que estan en s2
s1.difference_update(s2)
s1 -= s2
```

```
# Actualiza s1 con la diferencia simetrica entre s1 y s2
s1.symmetric_difference_update(s2)
s1 ^= s2
```

SortedSet puede hacer todo eso, pero esta ordenado y tambien tiene:

```
# pip install sortedcontainers -> Instalacion
from sortedcontainers import SortedSet
s = SortedSet([1, 2, 3, 4]) # Declarar
mini = s.pop() # Retorna el elemento menor
cond = 2 in s # Retorna True si s esta en s
i = s.index(2) # Retorna la posicion de 2 (0-index)
s.update([4, 5, 7]) # Agrega elementos
```

```
s.bisect_left(6) # Retorna el indice en que se insertaria el 6
s.bisect_right(3) # Lo mismo que bisect_left pero si el elemento
ya
```

esta en s entonces devuelve su posicion + 1

Diccionarios

```
dic = {'a': 1, 'b': 2} # Declarar
n = dic.get('a') # Acceder al value de la key 'a', sino esta
devuelve None
n = dic.get('a', 0) # Si la key 'a' esta, devuelve su value, sino
devuelve 0
claves = dic.keys() # Retorna una lista con todas las keys
valores = dic.values() # Retorna una lista con todos los valores
parejas = dic.items() # Retorna una lista de tuplas con todos
los pares key-value
dic.update({'c': 3, 'd': 4}) # Insertar elementos en dic
dic.update(dic2) # Actualizar dic con los elementos de dic2
a = dic.pop('a') # Elimina y retorna el value cuya key es 'a', sino
esta genera error
a = dic.pop('a', 0) # Elimina y retorna el value cuya key es 'a',
sino esta devuelve 0
a = dic.popitem() # Elimina y devuelve el ultimo par key-value
que fue insertado
dict.clear() # Elimina el diccionario
valor = dict.setdefault('a') # Si esta esa key, devuelve su value
valor = dict.setdefault('a', 2) # Si esta esa key, devuelve 2
value = dict.setdefault('e', 5) # Si no esta esa key, la agrega con
ese value y retorna el value
keys = ['a', 'b', 'c']
dic = dict.fromkeys(keys, 0) # Crea un nuevo diccionario con
claves del iterable y todas con el mismo valor
```



```
# Diccionarios ordenados
from sortedcontainers import SortedDict
dic = SortedDict({'c' : 3, 'a' : 1, 'b' : 2}) # Declarar
valor = dic['a'] # Devuelve el valor de la clave dada
dic['d'] = 4 # Asignacion
elemento = dic.popitem(0) # Elimina el elemento en la posicion
0, sino se especifica el indice elimina el ultimo elemento
# Tambien tiene bisect_left(key) y bisect_right(key), los cuales
funcionan igual que en SortedSet
pareja = dic.peekitem(1) # Devuelve el par clave valor en el
indice dado, sin eliminarlo
pareja = dic.popitem() # Elimina el ultimo elemento y lo retorna
pareja = dic.popitem(last=False) # Elimina el primer elemento y
lo retorna

main()
```

8. Algo de Teoría

8.1 Aplicaciones de Dinic:

/*

Aplicaciones:

- Maximo flujo

- Minimo corte:

Es lo mismo que el maximo flujo. El conjunto de nodos alcanzables (partiendo desde source) sin pasar por aristas saturadas es un lado del corte (left), y los no alcanzables son el otro lado del corte (right). Las aristas que pertenecen al mincut son aquellas que van de left a right y estan saturadas.

- Edge-disjoint paths:

Para hallar el maximo de caminos desde source hasta sink sin repetir aristas, se usa un flujo donde cada arista tiene capacidad 1. Para reconstruir estos caminos podemos guardar para cada nodo u una cola con los nodos v a los que este envia flujo, entonces mientras la cola de source no este vacia, tomamos el primer elemento v, nos movemos a la cola de v y sacamos el primer vertice x al que este envia flujo y asi sucesivamente hasta que llegamos a sink.

- Node-disjoint paths:

Se resuelve de la misma manera que Edge-disjoint paths pero dividiendo los nodos en u_{in} ($2*u$) y u_{out} ($2*u + 1$) y creando una arista con capacidad 1

entre ellos. Si hay una arista entre u y v, conectamos u_{out} con v_{in} .

- Minimum vertex cut

Para hallar el minimum vertex cut tendríamos que llamar Dinic con $2*n + 2$

y añadir una arista entre cada $2*i$ y $2*i + 1$ con capacidad de 1, para

$0 \leq i < n$. Despues se agregan aristas con capacidad 1 según las condiciones

del ejercicio o del grafo original. Si buscamos añadir una arista desde un

vertice u hasta un vertice v, usamos: `graph.addEdge(2*u + 1, 2*v, 1)`.

Despues agregamos aristas desde el nodo $2*n$ hacia todos los nodos iniciales

y desde todos los nodos finales hacia el nodo $2*n + 1$.

Finalmente, usamos `graph.calc(2*n, 2*n + 1)`, para hallar el maximo flujo

desde $2*n$ (que cumple la funcion de source-fuente), hasta $2*n + 1$ (que cumple

la funcion de sink-sumidero). El valor del maximo flujo corresponde al valor

del minimum vertex cut.

- Node-disjoint path cover en un DAG:

Un path cover es un conjunto de caminos en un grafo tal que cada nodo pertenece

a al menos un camino. Un Node-disjoint path cover quiere decir que cada nodo

debe pertenecer a exactamente un camino.

Para hallar el minimum Node-disjoint path cover podemos crear un grafo bipartito

donde separamos cada nodo u en u_{in} ($2u$)(right) y u_{out} ($2u + 1$)(left), entonces para cada arista u, v conectamos u_{out} con v_{in} . Despues hallamos el matching y esas son las aristas que vamos a elegir para armar el Node-disjoint path cover, entonces el total de caminos sera $n - c$, siendo n la cantidad de nodos en el grafo original y c la cantidad de aristas en el matching. Las aristas que se usan en los caminos de la particion son las aristas del matching, para saber donde empiezan los caminos, tomo los u tal que u_{in} no fue matcheado.

- General path cover en un DAG:

El minimum path cover se puede hallar de manera similar al minimum node-disjoint path cover, pero esta vez vamos a agregar una arista de u_{out} hacia v_{in} si en el grafo original se puede llegar desde u hasta v (posiblemente a traves de multiples aristas).

- Maximum Antichain:

Una antichain es un conjunto de nodos de un grafo en el cual ningun nodo tiene un camino hacia cualquier otro usando las aristas del grafo. En un DAG, la longitud de la maximum antichain es igual a la longitud del minimum path cover. Para reconstruirla hallamos el minimum vertex cover M , los nodos tales que u_{in} no pertenece a M y u_{out} tampoco, pertenecen a la maximum antichain.

- Maquinas y tareas (problema de asignacion):

Tengo un conjunto B de tareas que me dan un beneficio $B(t)$, puedo comprar un conjunto de maquinas M por un costo $M(t)$. Para realizar la tarea B tengo que comprar ciertas maquinas. Cada maquina la puedo utilizar para realizar cualquier cantidad de tareas. Para resolverlo asumo que ya me pagaron las tareas y tengo un beneficio total X , entonces no realizar una tarea tiene un costo $B(t)$. Construyo un grafo bipartito, conecto S con cada maquina con una capacidad $M(t)$, conecto cada tarea con T con una capacidad $B(t)$, y finalmente para cada tarea B y cada maquina M requerida para realizar la tarea B agrego una arista con capacidad infinita. Finalmente, el beneficio maximo es X menos el corte minimo. Si la arista S, M forma parte del corte, entonces compro la maquina M , si la arista B, T hace parte del corte, entonces no realizo la tarea B . En general, si tengo un problema que consiste en elegir un conjunto de nodos V' (donde cada nodo tiene un valor $w(v)$) tal que para toda arista u, v , v pertenece a V' si y solo si u tambien pertenece a V' , y se maximice el valor de los nodos elegidos, lo puedo resolver asi: agrego nodos S y T , agrego una arista S con capacidad $\sum w(v)$, v si $w(v) < 0$, una arista v, T con capacidad $w(v)$ si $w(v) > 0$, y todas las aristas u, v del grafo original

con capacidad inf. Los nodos elegidos (V') son los alcanzables desde S en la red residual, el

valor de V' es la suma de los $w(v)$ positivos menos el valor del corte mínimo.

- Si tengo un problema en el cual elegir determinado nodo hace que no pueda elegir otros, y

quiero elegir un conjunto de nodos que maximice la suma de $w(v)$ para cada v elegido, lo puedo

resolver separando los nodos en 2 lados U y V (para formar un bipartito) asegurandome que

podría elegir todos los nodos de alguno de los lados sin ningún problema (que los nodos

que pertenezcan al mismo lado no se restringen entre sí), agrego aristas S, u con capacidad $w(u)$

para cada u que pertenezca a U , agrego aristas v, T con capacidad $w(v)$ para cada v que pertenezca

a V , y aristas u, v con capacidad infinita para cada par de nodos tal que si elijo u no puedo

elegir v y viceversa. La respuesta es la suma $w(v)$ para cada v que pertenece al grafo, menos el valor del mínimo corte.

Para bipartitos:

- Maximum Matching:

Se conectan todos los nodos de un lado $left$ a un nuevo nodo $source$ y todos los

nodos de otro lado $right$ a un nuevo nodo $sink$, le hace que las aristas del grafo

original tengan dirección desde su correspondiente nodo en $left$ hacia su

correspondiente nodo en $right$ y finalmente se calcula el flujo. Las aristas de

$left$ hacia $right$ con flujo = 1 son las que harán parte del matching.

- Minimum vertex cover:

Es el mínimo conjunto de nodos tal que cada arista del grafo tiene al menos

uno de los sus nodos en dicho conjunto. El tamaño del minimum vertex cover es el mismo

del maximum matching. Para hallar cuáles vértices pertenecen podemos ver entre la lista

de adyacencia de $source$ cuáles nodos hacen parte de $right$ en el corte, además revisamos

para cada vértice de $left$ si este tiene a $sink$ en su lista de adyacencia, estos también

hacen parte del minimum vertex cover.

- Maximum independent set:

Es el mayor conjunto de nodos tal que no hay 2 nodos del conjunto conectados por

una arista. Todos los nodos que no hacen parte del minimum vertex cover hacen parte

del maximum independent set, por lo tanto el tamaño de este es el total de nodos

del grafo menos el tamaño del maximum matching.

*/

8.2 Algunas aplicaciones de Dijkstra

// Algunas aplicaciones:

/*

Tambien sirve para grafos con pesos negativos siempre y cuando no haya ciclos negativos, en ese caso usar Bellman-Ford. Cabe resaltar que si el grafo es grande, es propenso a dar TLE.

*/

/*

Para saber las k menores distancias desde s hasta t (usando k caminos diferentes)

podemos modificar el vector de distancias, haciendo que sea un vector de colas de

prioridad. Asi cuando llegamos para cada nodo almacenamos las k distancias menores.

Entonces cuando sacamos un nodo u de la cola de prioridad del dijkstra, verificamos

si su peso mayor en su cola de prioridad es menor que el que estamos analizando en

el momento, en caso de que si, continuamos con la siguiente iteracion. Luego, para

cada nodo adyacente v, verificamos si este aun no tiene k elementos en su cola de

prioridad y agregamos el nuevo peso tanto en su cola de prioridad como en la del

dijkstra. En caso de que si tenga k elementos, verificamos si la distancia w_1 mas

el peso de la arista que estamos analizando es menor que el valor mayor de la cola

de prioridad de v, en caso de que si, sacamos el valor mayor de la cola de prioridad

de v y ponemos esta nueva distancia, ademas de agregarla a la cola de prioridad del dijkstra.

La complejidad es $O(m \cdot k \cdot \log(n \cdot k))$, por lo tanto $m \cdot k$ debe ser pequeño.

Link de un ejercicio: <https://cses.fi/problemset/task/1196/>

*/

/*

Podemos hallar el potencial de un grafo:

En caso de que el grafo tenga pesos negativos, podemos hallar una funcion p dado que

para cada arista $(u \rightarrow v)$, $w(u \rightarrow v) + p[u] - p[v] \geq 0$.

La podemos hallar creando un nuevo vertice s y aristas que vayan desde s hacia todos

los vertices del grafo con peso 0. Entonces corremos dijkstra desde ese vertice y las

distancias desde s hacia cada vertice seran los valores de la funcion p (todos estos

seran ≤ 0). Ahora lo que hacemos es reemplazar el peso w de cada arista $u \rightarrow v$

por $w + p[u] - p[v]$. Basicamente la idea consiste en que a cada arista que salga desde

u le vamos a sumar $p[u]$ y a cada arista que llegue a u le vamos a restar $p[u]$.

Si bien la funcion potencial esta pensada para utilizarla en el algoritmo de Jhonson y

para min cost max flow, tambien se puede utilizar para mejorar la complejidad del dijkstra

en algunos casos específicos. Lógicamente no tiene sentido pensar en optimizar el dijkstra utilizando la función potencial, la cual es calculada usando el dijkstra, sin embargo en algunos ejercicios podemos darnos cuenta de que se puede calcular una función p válida sin necesidad de correr ningún algoritmo, en tal caso, ya podríamos hacer un dijkstra sobre un grafo sin pesos negativos, lo cual será mucho más eficiente.

Ejercicio ejemplo:

<https://codeforces.com/contest/1627/problem/E>

Ahora, para saber la distancia desde u hasta v , usando los pesos del grafo original, corremos el dijkstra desde u y la respuesta es $d[v] - p[u] + p[v]$, debido a que a cada vértice x intermedio le sumamos $p[x]$ al llegar y le restamos $p[x]$ al salir, pero en ningún momento le habíamos restado $p[u]$, debido a que partimos desde ahí, y tampoco habíamos sumado $p[v]$, debido a que nunca salimos de ahí.

Source: <https://codeforces.com/blog/entry/95823>

*/

/*

Para saber si una arista puede estar en un camino más corto desde s hasta t podemos hacer un dijkstra desde s y desde t , después recorrer cada arista u, v y revisar si la distancia de s hasta u , más la distancia de t hasta v , (o la

distancia desde s hasta v más la distancia desde t hasta u si el grafo es no dirigido) más el peso de esa arista es igual a la distancia desde s hasta t ,

en caso de que sí, quiere decir que esa arista puede hacer parte de un camino más corto desde s hasta t .

También se puede hacer para grafos dirigidos. En ese caso, el dijkstra desde t

tiene que hacerse invirtiendo todas las aristas del grafo.

Si armamos un grafo con las aristas que puedan estar en un camino más corto de

s hasta t , obtendríamos un DAG (Directed Acyclic Graph).

*/

/*

Para saber si una arista está en todos los caminos más cortos desde s hasta t

(que para llegar de s hasta t de la manera más eficiente posible si o si tengo

que pasar por cierta arista) podemos armar un grafo auxiliar con las aristas

que pueden estar en un camino más corto de s hasta t . Lo podemos armar como si

fuera un grafo dirigido teniendo en cuenta la distancia de s a los vértices u y

v de la arista. Después recorreremos el grafo con un BFS de la siguiente manera:

*/

```
void BFS(int s, int t, int n, vector<bool>& ans, map<pair, int>& edges, vector<vector<pair>>& graph, vector<int>& inDegree){
```

```

// ans está inicializado con 0s
vector<bool> visited(n+1); visited[s] = 1;
queue<int> q; q.push(s);
int roads = 1;
while(!q.empty()){
    int u = q.front(); q.pop();
    if(u != t) roads += graph[u].sz - 1; // Aumentamos roads con el
numero de salidas de u, - 1
    for(pil edge : graph[u]){
        int v = edge.first; ll w = edge.second;
        int index = edges[{{u, v}, w}]; // Para acceder al indice de la
arista;
        // tener cuidado por si la arista en el grafo
fuera (v, u) en lugar de (u, v)
        if(roads == 1) ans[index] = 1; // Si roads es 1, esa arista está
en todos los caminos más cortos de s a t

        // Despues de visitar v por primera vez, cada que lo vuelva a
visitar disminuyo roads
        if(visited[v]) roads--;
        visited[v] = 1; // Marco que ya lo visite por si es la primera
vez
        inDegree[v]--; // Resto su número de aristas de entrada
        if(!inDegree[v]) q.push(v); // Cuando he visitado la ultima
arista que entra en v, lo meto en la cola
    }
}
}

```

8.3 Heuristica para optimizar el SPFA

// Implementacion usando mcfx optimization:

/*

Esta implementacion utiliza "mcfx optimization", la cual consiste en utilizar una

deque en lugar de una queue, antes de agregar cada vertice revisamos si este ha

sido procesado entre 2 y $\sqrt{n}$ veces, le damos prioridad a visitarlo y por ende

lo agregamos al inicio de la deque. En cambio, si solo ha sido procesado una vez,

no hay nada raro con este, y cuando ha sido visitado muchas veces (mas de $\sqrt{n}$),

posiblemente sea un "bad vertex" o "trick vertex", el cual nos lleva a un camino

suboptimo y por ende en estos 2 casos, no le damos prioridad a volver a visitar a

este vertice y lo agregamos al final de la deque. Si bien esta optimizacion tambien

corre en $O(n * m)$, en muchos casos logra correr mas rapido, evitando el TLE.

- Suele funcionar muy bien en grid graphs (grafos tipo cuadro)
- No funciona bien en star graphs (un nodo central y $n-1$ aristas, las cuales conectan

los nodos perifericos con el nodo central)

*/

bool spfa(int s, int n, vector<ll>& d){

for(int i = 1; i <= n; i++) d[i] = inf;

d[s] = 0;

vector<int> count(n+1);

vector<bool> in(n+1);

in[s] = 1;

deque<int> q;

q.push_front(s);

int mini = 2, maxi = $\sqrt{n}$ + 5;

while(!q.empty()){

int u = q.front(); q.pop_front();

in[u] = 0; // Marca que el vertice ya no esta en la cola

for(pil edge : graph[u]){

int v = edge.fi; ll w = edge.se;

if(d[v] > d[u] + w){

d[v] = d[u] + w;

if(!in[v]){ // Antes de agregar un vertice a la cola, verifica que no se encuentre ya en ella

in[v] = 1; // Marca que el vertice ahora esta en la cola

if(count[v] >= mini && count[v] <= maxi) q.push_back(v);

else q.push_front(v);

count[v]++;

if(count[v] > maxi)

return 0; // Si el nodo ha sido metido en la cola más de n veces, hay un ciclo negativo

}

}

}

}

return 1; // No hay ciclos negativos

}

8.4 Aplicaciones Segment Tree (Updates individuales)

En esencia solo hay que cambiar los operadores o usar la técnica “Walking on a Segment Tree”

8.4.1 Contar la cantidad de veces que se repite el valor mínimo

```
pii compMin(pii a, pii b){
    if(a.fi == b.fi) return {a.fi, a.se + b.se};
    return min(a, b);
}
```

8.4.2 Intersecting Segments (Oper suma)

Para cada segmento cuenta la cantidad de otros segmentos con los cuales

intersecta.

Los recorre en orden del primero en r (que termina primero).

Cuando llega

a l de algun segmento, asigna un 1 en esa posicion, cuando llega a r asigna

la posicion l en -1 y r en 1. Para saber la cantidad pregunta la suma en el

[l, r].

Todos los puntos de los segmentos son diferentes entre si.

```
void solver(){
    int n; cin>>n;
    int n2 = 2*n;
    vector<int> a(n2);
    for(int i = 0; i < n2; i++){cin>>a[i]; a[i]--;}

    vector<int> ans(n), st(n, -1);
    for(int i = 0; i < n2; i++){
```

```
        int num = a[i];
        if(st[num] == -1){ // Si no he tocado el extremo izquierdo, lo
            guardo, asigno 1 en l y sigo
            st[num] = i;
            update(1, 0, n2 - 1, i, 1);
            continue;
        }
```

```
        int l = st[num];
        update(1, 0, n2 - 1, l, -1); // Asigno -1 en l
        update(1, 0, n2 - 1, i, 1); // Asigno 1 en r
        ans[num] = query(1, 0, n2 - 1, l, i); // Pregunto por la suma
        desde l hasta i
    }
```

```
    for(int i = 0; i < n; i++){
        if(i) cout<<' ';
        cout<<ans[i];
    }
    cout<<endl;
}
```

8.4.3 Nested Segments (Oper suma)

Dados ciertos segmentos, los recorre en orden del primero en r (que termina primero) y dice la cantidad de segmentos que contiene (si estoy analizando un segmento x1, x2, cuenta la cantidad de segmentos y1, y2 con $x1 < y1 \leq y2 < x2$).

Todos los puntos de los segmentos son diferentes entre si.

```
void solver(){
    int n; cin>>n;
    int n2 = 2*n;
```

```

vector<int> a(n2);
for(int i = 0; i < n2; i++){cin>>a[i]; a[i]--;}

vector<int> ans(n), st(n, -1);

for(int i = 0; i < n2; i++){
    if(st[a[i]] == -1){ // Si no he tocado el extremo izquierdo, lo
        guardo y sigo
        st[a[i]] = i;
        continue;
    }

    int num = a[i];
    int l = st[num];
    ans[num] = query(1, 0, n2 - 1, l, i); // Pregunto por la suma
    desde l hasta i
    update(1, 0, n2 - 1, l, 1); // Cuando termino de analizar un
    segmento, pongo su l en 1
}

for(int i = 0; i < n; i++){
    if(i) cout<<' ';
    cout<<ans[i];
}
cout<<endl;
}

```

8.4.3 Kth one

// Si tengo un array con todo 1s, me va a decir que el 4to uno
 // esta en la posicion 3. Asi que si me piden que diga el Kth
 // uno en 0-index, es decir, donde el uno en la posicion 0
 // es el primero, tengo que sumar 1 a k para cada query.

```

// Si me piden que diga el Kth uno en 1-index, puedo usar
// el valor normal de k
// - Sirve para ejercicios donde tenga que activar y desactivar
// posiciones, y contestar queries del tipo de cual es la Kth
// posicion encendida
const int maxn = 1e5 + 1;
vector<pii> tree(4*maxn);

void build(vector<int>& a, int v, int tl, int tr){
    if(tl == tr) tree[v] = {a[tl], tl};
    else{
        int tm = (tl + tr) / 2;
        build(a, 2*v, tl, tm);
        build(a, 2*v + 1, tm + 1, tr);
        tree[v] = {tree[2*v].fi + tree[2*v + 1].fi, 0};
    }
}

// Las queries son [l, r]
int query(int v, int tl, int tr, int k){
    if(tl == tr) return tree[v].se;

    int tm = (tl + tr) / 2;
    if(tree[2*v].fi >= k) return query(2*v, tl, tm, k);
    return query(2*v + 1, tm + 1, tr, k - tree[2*v].fi);
}

void update(int v, int tl, int tr, int pos, int val){
    if(tl == tr) tree[v] = {val, pos};
    else{
        int tm = (tl + tr) / 2;
        if(pos <= tm) update(2*v, tl, tm, pos, val);
    }
}

```

```

        else update(2*v + 1, tm + 1, tr, pos, val);
        tree[v] = {tree[2*v].fi + tree[2*v + 1].fi, 0};
    }
}

```

```

void solver(){
    int n, m; cin>>n>>m;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin>>a[i];

    build(a, 1, 0, n-1);

    while(m--){
        int op; cin>>op;
        if(op == 1){
            int i; cin>>i;
            a[i] = 1 ^ a[i];
            update(1, 0, n-1, i, a[i]);
        }
        else{
            int k; cin>>k; k++;
            cout<<query(1, 0, n-1, k)<<endl;
        }
    }
}

```

8.4.4 Primer elemento mayor a x

/*

Sirve para hallar el primer j tal que $j > l$ y $a[j] > x$.
 Si simplemente se quiere hallar el primer indice en cualquier parte del array, se envia a la funcion query un valor de $l = 0$.

*/

```
const int maxn = 1e5 + 1;
```

```
vector<pii> tree(4*maxn);
```

```

void build(vector<int>& a, int v, int tl, int tr){
    if(tl == tr) tree[v] = {a[tl], tl};
    else{
        int tm = (tl + tr) / 2;
        build(a, 2*v, tl, tm);
        build(a, 2*v + 1, tm + 1, tr);
        tree[v] = max(tree[2*v], tree[2*v + 1]);
    }
}

```

// Las queries son [l, r]

```

int query(int v, int tl, int tr, int x, int l){
    if(l > tr) return -1;
    if(tl == tr){
        if(tree[v].fi < x) return -1;
        return tree[v].se;
    }
}

```

```

    int tm = (tl + tr) / 2;
    int ans = -1;
    if(tree[2*v].fi >= x) ans = query(2*v, tl, tm, x, l);
    if(ans == -1 && tree[2*v + 1].fi >= x) ans = query(2*v + 1, tm + 1,
tr, x, l);
    return ans;
}

```

```

void update(int v, int tl, int tr, int pos, int val){
    if(tl == tr) tree[v] = {val, pos};
    else{
        int tm = (tl + tr) / 2;
        if(pos <= tm) update(2*v, tl, tm, pos, val);
        else update(2*v + 1, tm + 1, tr, pos, val);
        tree[v] = max(tree[2*v], tree[2*v + 1]);
    }
}

```

```

void solver(){
    int n, m; cin>>n>>m;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin>>a[i];

```

```

    build(a, 1, 0, n-1);

```

```

    while(m--){
        int op; cin>>op;
        if(op == 1){
            int i, v; cin>>i>>v;
            update(1, 0, n-1, i, v);
        }
        else{
            int x, l; cin>>x>>l;
            cout<<query(1, 0, n-1, x, l)<<endl;
        }
    }
}

```

8.5 Teoría de Juegos

Números de Grundy:

- Si el número de Grundy de una posición es 0, es una posición perdedora.
- Si es mayor a 0, es una posición ganadora.

Se calcula calculando el mex (menor número excluido) de los números de Grundy de las posiciones a las que me puedo mover.

Ejemplo: Juego de Nim

- Se calcula el XOR del tamaño de todas las pilas
- Si el resultado es mayor a 0, gana el jugador que arranca.
- Si no, gana el segundo jugador.
- Si solo puedo remover 1, 2 o 3 palos, toca aplicarle mod 4 al tamaño de cada pila, antes de hacer el XOR.